OPTIMIZING DATA STRUCTURES WITH ALTERNATING-ORIENTED BISECTED LAYOUTS

Aleksandar Tucovic

October 29, 2018

1 OVERVIEW

Layouts are a general and universal design tool. However, for the purposes of modern computing layouts are lacking a general and universal data structure. In this paper, a layout data structure will be described and applied to various areas of computer science. It will be proposed that there is one data structure that can be thought of as a general theory of layouts.

The XOO layout schema is a general-purpose and scalable tool for design logic. For the purposes of determining and optimizing data structures, the XOO layout schema is constrained by bisecting divisions of the layout in alternating orientations.

There has been some interesting research in layout and applications of layout structure over the years but with a little bit of insight it extends into many areas of computer science. One of the interesting properties of the XOO schema in applying a loose and strict set of rules to its data structure is that interesting geometries appear, and that there are overlaps in many areas of computer science.

1.1 APPLICATIONS

There are a number of areas where this layout engine can be applied:

- Design logic
- Circuit design
- Parallel computing
- Distributed computing
- Distributed networking
- Software development
- Manufacturing
- Architecture

16	18	24	26
17	19	25	27
20	22	28	30
21	23	29	31

Figure 1: 4D Dense Layout

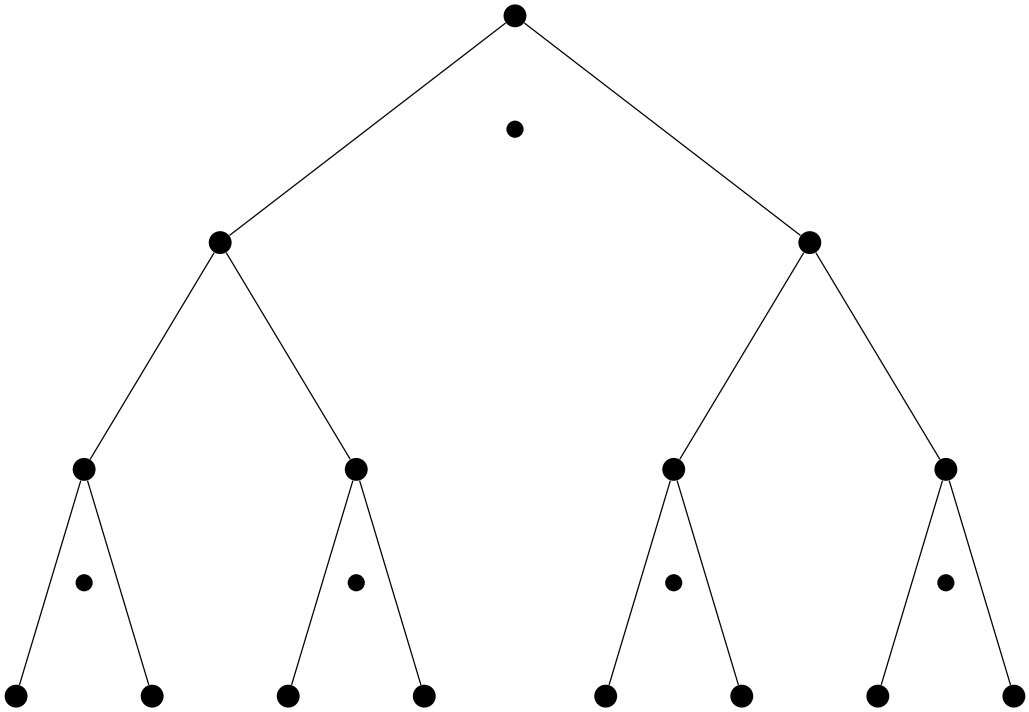


Figure 2: Binary Hypertree

1.2 IMPLEMENTATION

To begin, a general layout data structure is described. This is done in an iterative approach. This is as specified in "a method for users to create and edit web page layouts."

Begin with a rectangular region R_1.

Bimedians H or V bisects the region R_1 into equal subregions R_2 and R_3. Only one bimedian is allowed per region and bimedians alternate between H and V in each subregion relative to its region.

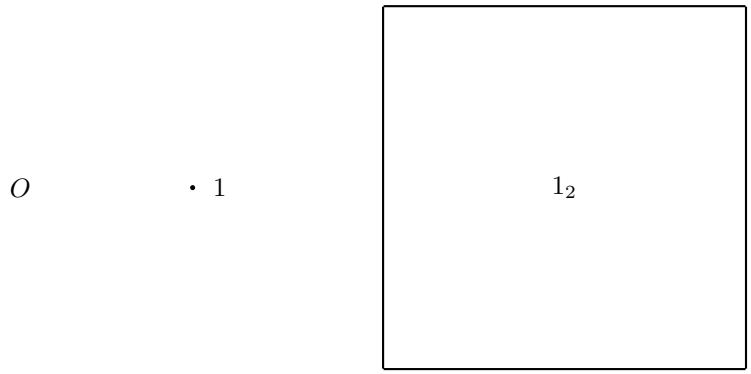
In general,

$$R_i \to R_{2i} \cup R_{2i+1}$$

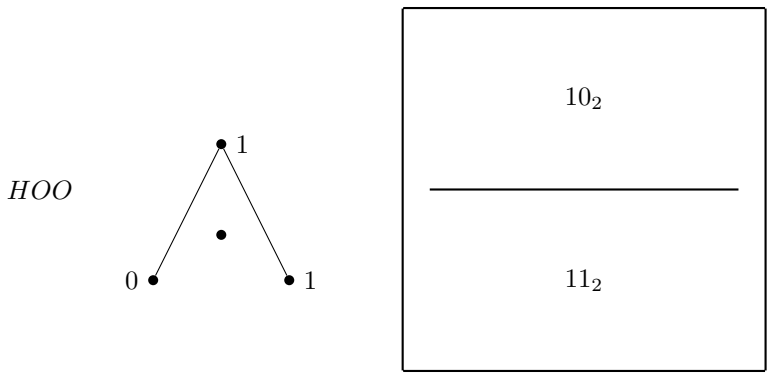
and

$$R_{2i} \cap R_{2i+1} = \emptyset$$

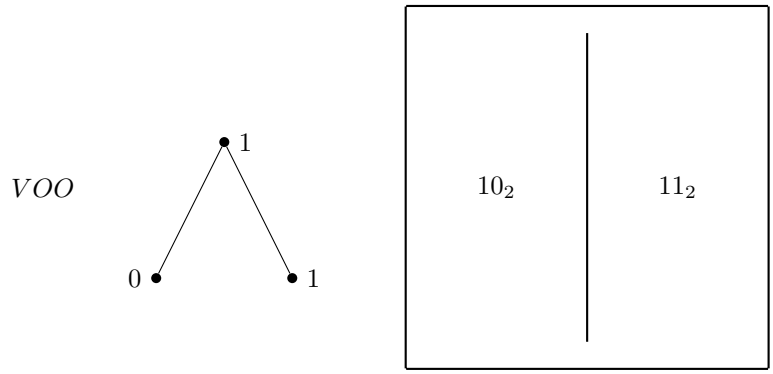
The region R_1 is also known as a topological flat torus.



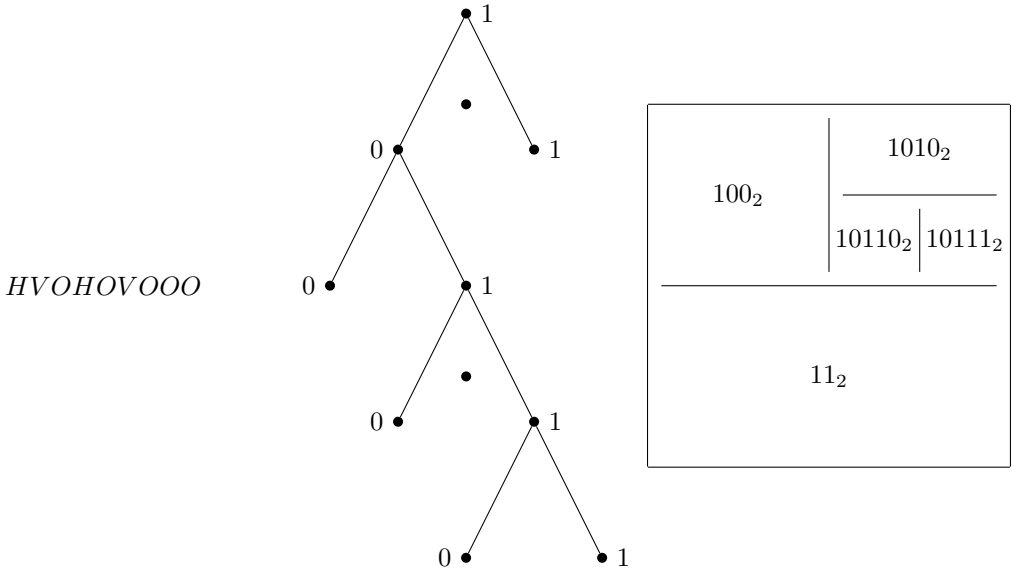
After the initial region is created then it can be bisected horizontally or vertically.



The above is horizontally bisected. The below is vertically bisected.



The above illustrates a general, open-ended approach to layouts. Each subdivision of a layout can only be bisected either horizontally or vertically.



Start with an empty layout, initially O, labeling the subdivisions by its binary tree encodings.

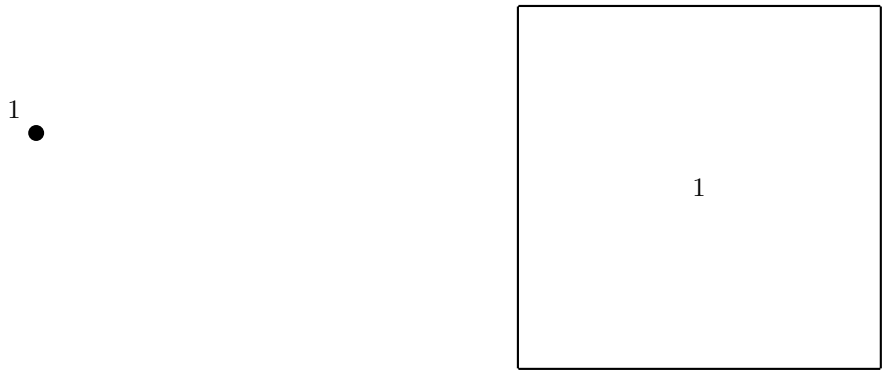
Now let's start graphing the layout in such a way:

Subdivisions are bisected alternating between horizontal and vertical orientations;

Each subdivision is exactly half of its parent in the orientation of the bisection;

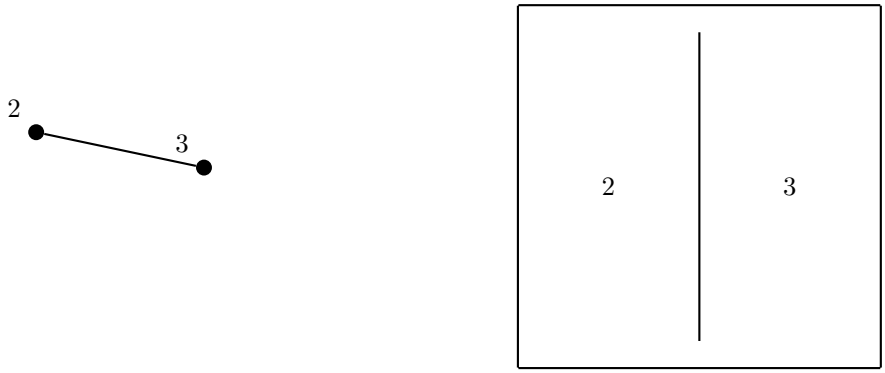
Each subdivision will be represented by a graph vertex;

A graph edge will represent neighbourliness of subdivisions as represented by the layout displayed.

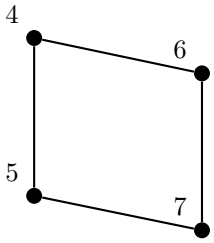


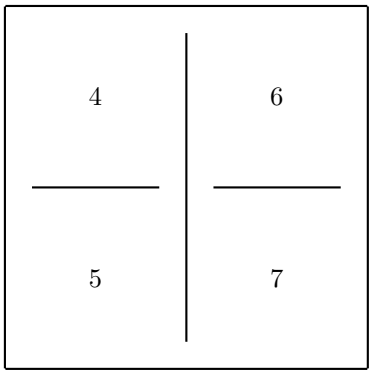
The initial division has no neighbours so there is only one vertex and no edges. This is zero dimensions (0D)

Next, subdivide the layout, without loss of generality, into VOO and label the subdivisions according to the XOO layout schema. Be sure to subdivide each division only once and when this is done, this is called a dense layout. No more divisions can be subdivided without going up a dimension. This is a 1D layout and graph:

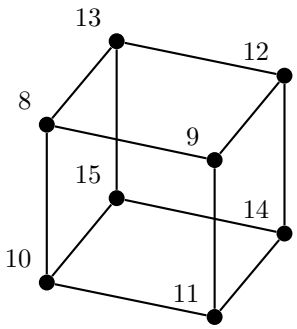


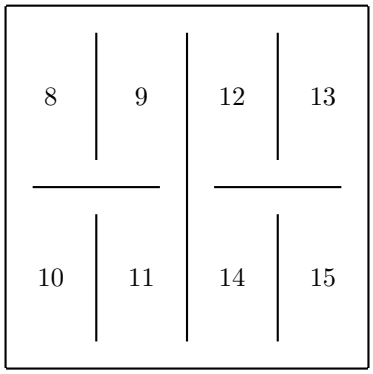
Next, subdivide the 1 Dim layout, this time replacing each O with HOO. This dense layout is then 2D:





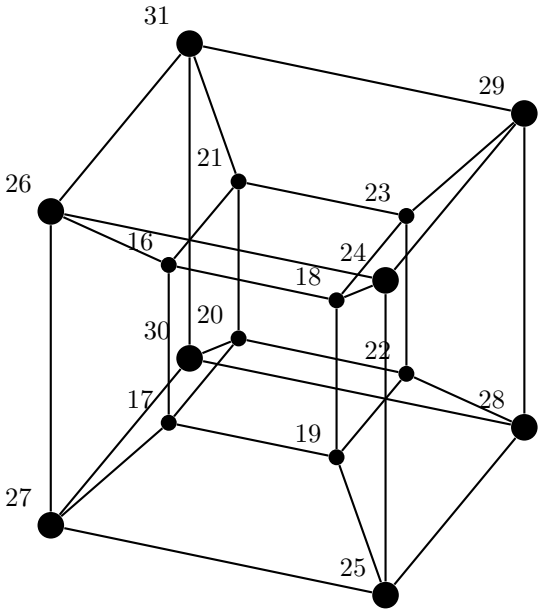
Continue on alternating between VOO and HOO subdivisions and the overall dense layout grows by one dimension:





As can be seen, graphing the layout in such a way as having subdivisions represented by vertices and neighbours representing edges, it begins to represent a 3D cube. This means that a two-dimensional XOO layout can represent a n-dimensional graph.

Continuing with the next subdivision, a 4D layout and graph are represented:



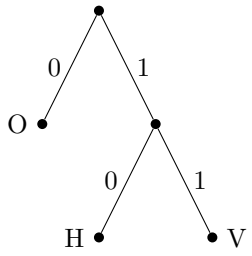
16	18	24	26
17	19	25	27
20	22	28	30
21	23	29	31

This continued subdivision of dense layouts represents what are called hypercubes.

This extension of "a method for users to create and edit web page layouts" includes an implementation regarding the encoding of the XOO. Essentially, the XOO is a specialized ternary encoding. The three characters of O, V and H can be encoded in binary using Huffman encoding as a basis. So, the most frequent character O represents approximately half of all the XOO encoding so its encoded as 0_2 (zero). The character V and H appear equally frequently so will be represented as 10_2 and 11_2 respectively.

The XOO encoding can be binary encoded and "a method for users to create and edit web page layouts" [85] is extended to include encodings, layouts, binary trees and graphs (user interface is not illustrated):

$O = 0_2$
$V = 11_2$
$H = 10_2$



These encodings are equivalent:

$$HVOHOVOOO$$
$$-+0-0+000$$
$$0001011010011000_2 = 0x1698$$

The benefit of the binary encoding over the human-readable ASCII XOO encoding is compression - only ternary encoding is more compact and efficient than binary encoding. The encoded XOO format can then be used to store or communicate sophisticated data structures very efficiently.

Another benefit of this data structure is that data structures can be free of variable names, and all of the structure and node information can be extracted from the encoding.

1.3 DESIGN LOGIC

A method for users to create and edit web page layouts describes how layouts are encoded, represented and displayed with a user interface for editing. It is extended as above using graphs to represent hypercube architectures and various applications as below.

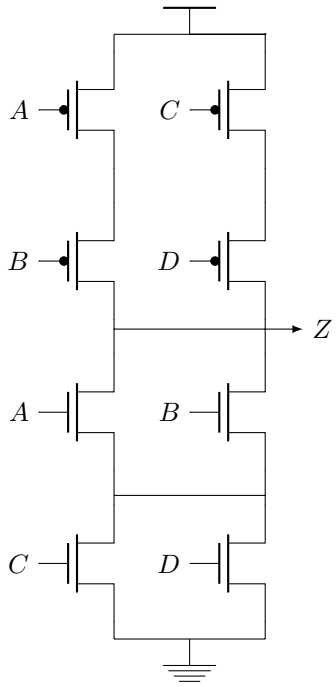
1.4 CIRCUIT DESIGN

A state machine is being designed with the goal of creating specialized circuits that can read/write/verify XOO-formatted data up to some k-dimension.

In circuit design, Karnaugh maps are an established way of simplifying truth tables representing circuits. According to Wikipedia,

Karnaugh maps are used to simplify real-world logic requirements so that they can be implemented using a minimum number of physical logic gates.

Starting with a simple 4-input circuit:



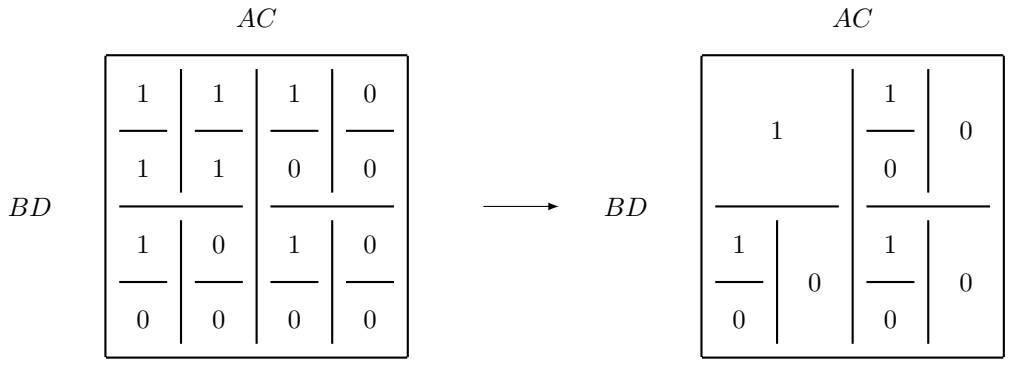
The truth table representing the circuit logic:

A	B	C	D	Z
0	0	0	0	1
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	1
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

The Karnaugh map, or K-map, is a tabular representation of the logic table. The K-map implements Gray codes as part of the tabular representation. In contrast, using the XOO layout (XOO-map) eliminates the need for explicit Gray codes and eliminates the need to look for groups of 1s wrapping around the K-map.

Using the XOO-map requires only identification of groups of 1s and any symmetries horizontally and vertically (but not diagonally). The XOO-map can merge similarly adjacent 1s and 0s, and group those together in larger subdivisions. Then finally, reading off the XOO-map provides the simplified sum-of-products. Additionally, it can also be represented variable-free in the XOO encoding.

Heres the alternative XOO-map as defined by the above truth table:



Reading the XOO-map takes some practice. Note that $\overline{A}$ equals $NOTA$, $\overline{B}$ equals $NOTB$, etc. Inputs alternate in a binary fashion:

By group, read the first upper-left quadrant group of 1s as $\overline{AB}$ because the group spans only $\overline{A}$ and $\overline{B}$. (It also spans C and $\overline{C}$, D and $\overline{D}$, so are ignored.) By symmetry, read the upper-left 1s of

each quadrant as $\overline{CD}$ because the group spans only $\overline{C}$ and $\overline{D}$. (It also spans A and $\overline{A}$, B and $\overline{B}$, and so are ignored.)

Therefore, the sum-of-product is:

$$\overline{AB} + \overline{CD}$$

Without loss of generality, the XOO-encoded, variable-free, sum-of-product is:

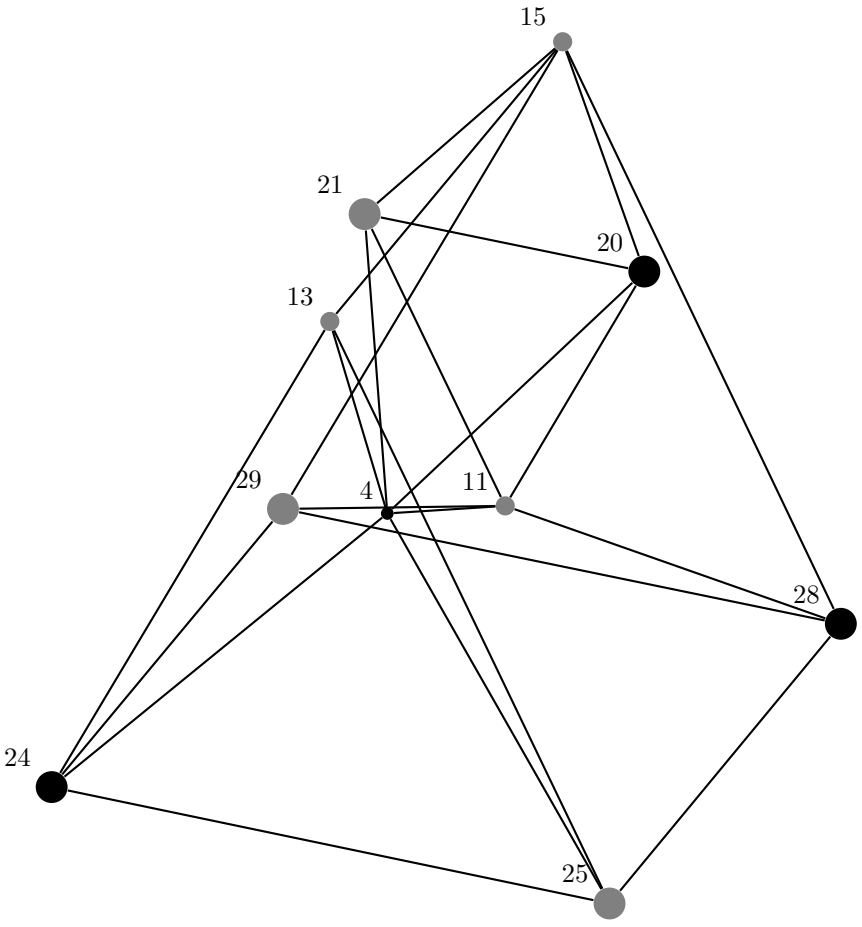
$$VHOVHOOOHVHOOOVHOOO, 1, 1, 0, 0, 1, 0, 0, 1, 0, 0$$

or (where H = 10_2, V = 11_2 and O = 0_2),

$$0x39C2E1C324$$

The encoding is 40 bits. The original truth table is encoded in 5*16 = 80 bits (not including the labels.)

For the curious, the hyper-object of the above sum-of-products is a sub-hypercube:

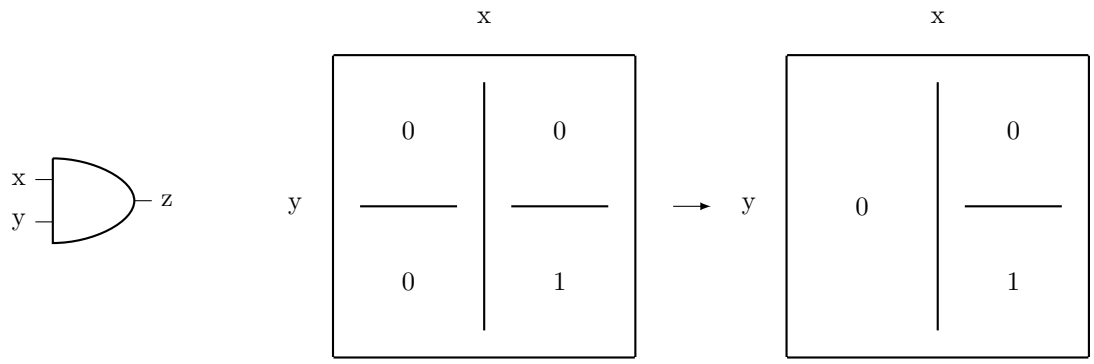


1.5 LOGIC GATES

Logic gates are easily interpreted in the XOO layout producing some simplified logic tables.

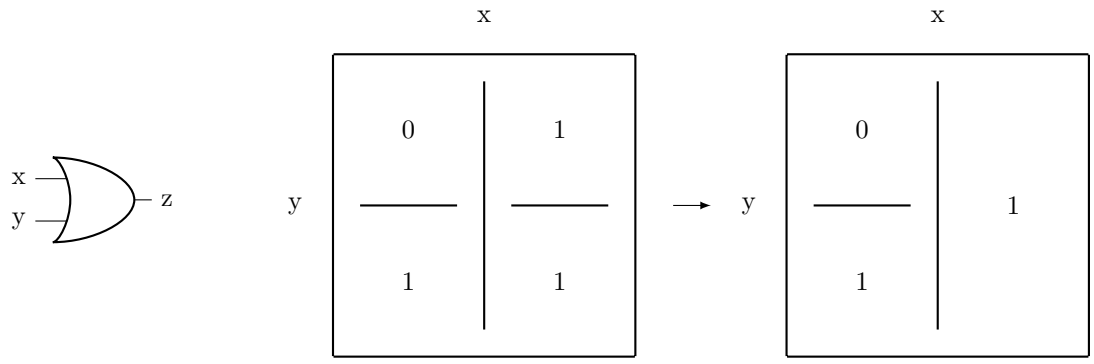
The AND gate is simplified to 10 bits:

$$VOHOO, 0, 0, 1$$



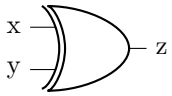
The OR gate is 10 bits:

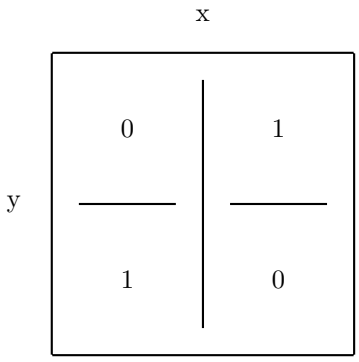
$$VHOOO, 0, 1, 1$$



The XOR gate is represented by 14 bits:

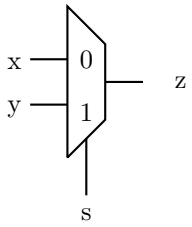
$$VHOOHOO, 0, 1, 1, 0$$

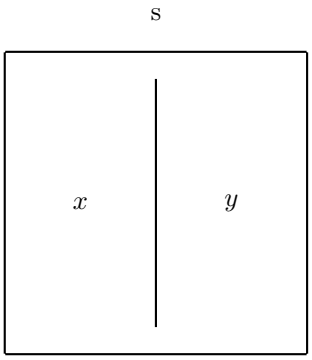




The multiplexer is represented by:

$$VOO, x, y$$



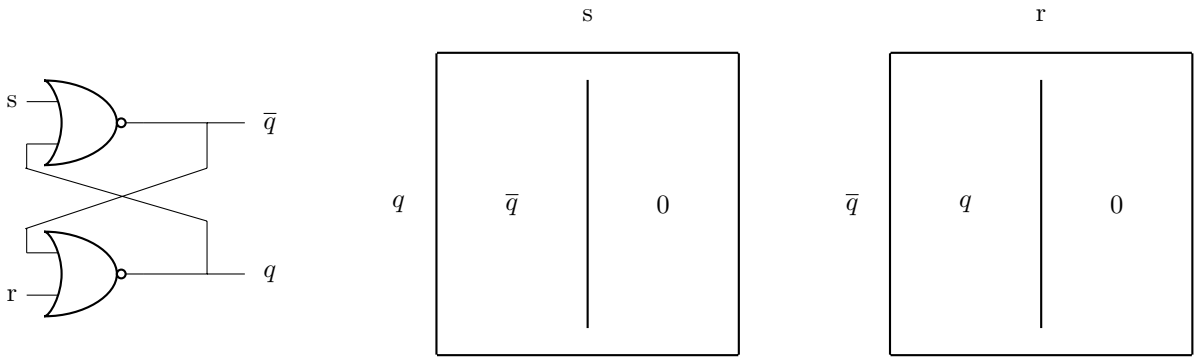


The SR-Latch is as follows:

$$VOO, \overline{q}, 0$$

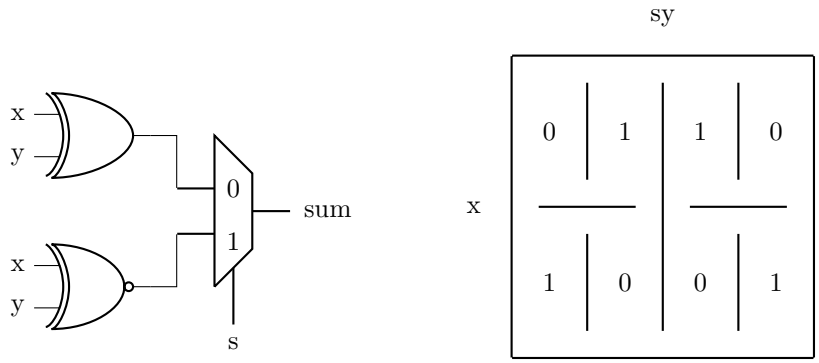
and

$$VOO, q, 0$$



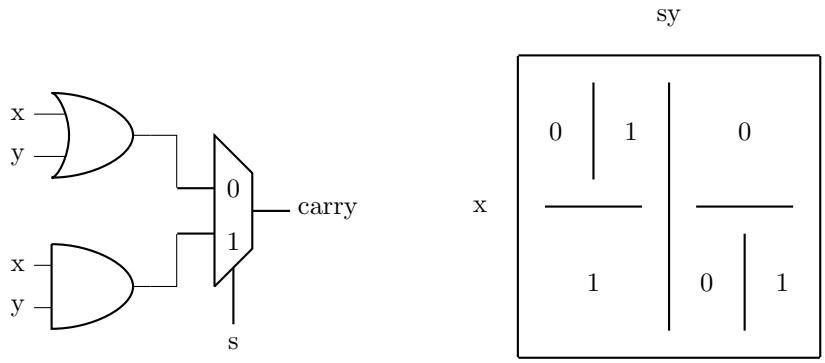
The full adder is composed of the sum:

$$VHVOOVOOHVOOVOO, 0, 1, 1, 0, 1, 0, 0, 1$$



and carry:

$$VHVOOOHOVOO, 0, 1, 1, 0, 0, 1$$



1.6 TERNARY- (TRINARY-) BASED LOGIC

The XOO notation allows three values for bisecting layouts: a 'H' or a 'V' for horizontal and vertical bisecting respectively and an 'O' to illustrate a node.

Ternary is a base three numeral system using three digits: 0, 1, 2.

The balanced ternary notation uses the notation for values negative one, zero and positive one: -, 0, +. The balanced ternary notation will be used here.

The XOO notation implies a ternary numbering system. Ternary logical tables suggest that there may be a way to represent logic using the same XOO layout system. Unary and binary logical operators follow for ternary values:

x	NOT(x)
-	+
0	0
+	-

AND	-	0	+
-	-	-	-
0	-	0	0
+	-	0	+

The challenge is to derive a XOO layout that can represent the logical operators, sum-of-products and XOO encoding. The layout below is a general-purpose layout for ternary operators:

x

y	0	−	0
	−	−	−
	+	−	+

The difference between the ternary layout and previous binary layouts is that in the ternary layout the subdivisions are bisected between the zero and the nonzero values, and the nonzero value is then bisected between the negative and positive values. The bisection of the ternary layout does not alternate, either. The XOO encoding is:

$$VOHOHOOVHOHOOHOHOO, 0, -, +, 0, -, +, 0, -, +$$

Where V is +, H is - and O is 0:

$$+0-0-00+-0-00-0-000-+0-+0-+$$

In addition to the above stipulations for ternary layouts, a special notation is introduced because the outputs can take two possible values that contribute to a sum-of-products. This is where ternary layouts differentiate from the above alternating bisected layouts but it is covered as it provides a completeness to the overall subject area.

$$|x| = \begin{cases} + & \text{if } x = - \\ Don't\ Care & \text{if } x = 0 \\ + & \text{if } x = + \end{cases}$$

$$[x] = \begin{cases} - & \text{if } x = - \\ DC & \text{if } x = 0 \\ + & \text{if } x = + \end{cases}$$

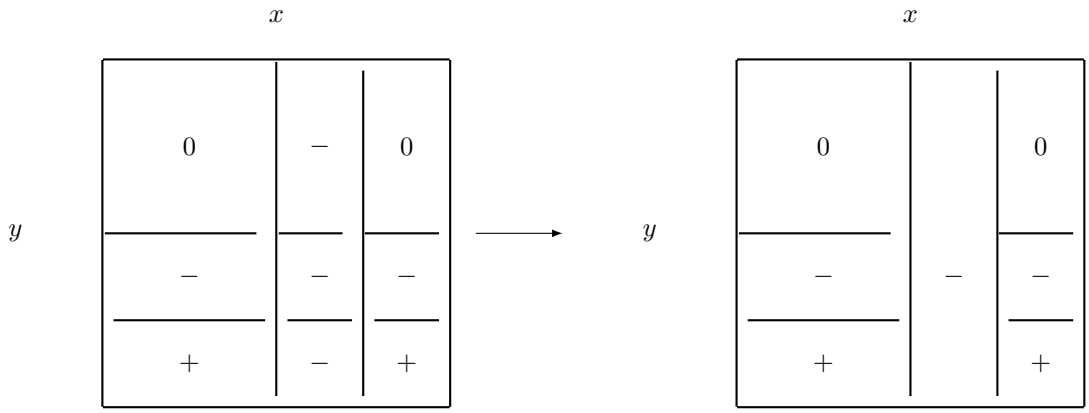
$$\lfloor x \rfloor = \begin{cases} - & \text{if } x = - \\ DC & \text{if } x \neq - \end{cases}$$

$$\lceil x \rceil = \begin{cases} + & \text{if } x = + \\ DC & \text{if } x \neq + \end{cases}$$

$$\overline{x} = \begin{cases} + & \text{if } x = 0 \\ DC & \text{if } x \neq 0 \end{cases}$$

$$\tilde{x} = \begin{cases} + & \text{if } x = - \\ DC & \text{if } x = 0 \\ - & \text{if } x = + \end{cases}$$

Taking the AND logical table above and simply transferring the values to the left layout below, subdivisions can be merged to show subdivisions spanning 'y'. Then values are read off the simplified layout using the previous notation. The second column is '-1' so it is '$\lfloor$x$\rfloor$'. The second row is '$\lfloor$y$\rfloor$'. Finally, the intersection of the third column and third row is '$\lceil$x$\rceil\lceil$y$\rceil$'.



The sum-of-products is:

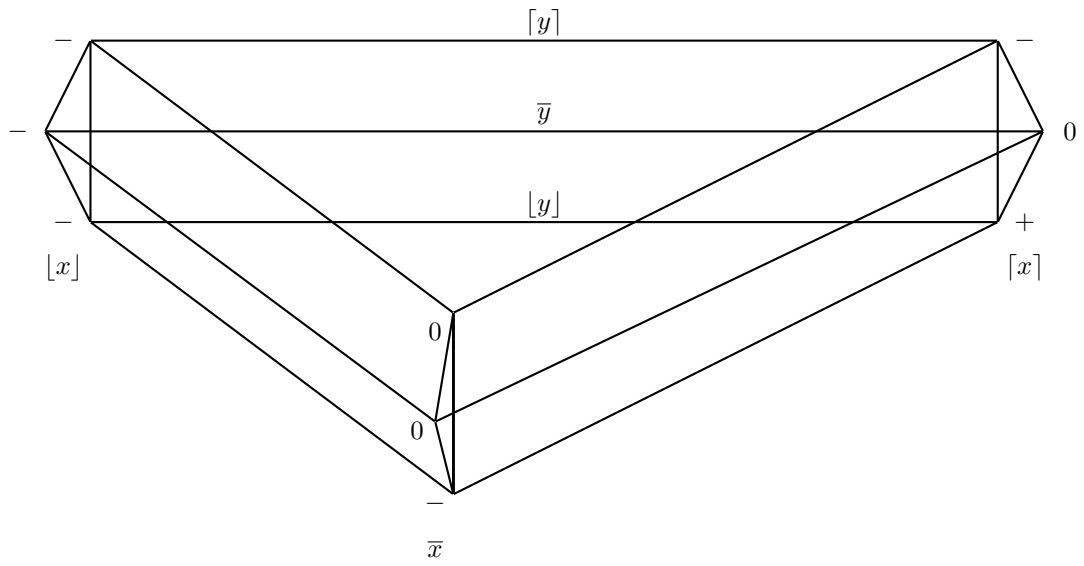
$$x \text{ AND } y = min(x, y) = \lfloor x \rfloor + \lfloor y \rfloor + \lceil x \rceil \lceil y \rceil$$

The ternary XOO encoding is:

$$VHOHOOVOHOHOO, 0, -, 0, -, 0, -, +$$

Or:

$$+ - 0 - 00 + 0 - 0 - 000 - 0 - 0 - +$$



1.7 GRAMMAR-BASED CODES

Suppose a XOO-encoding is given. A sub-encoding can be used to represent a portion of the original encoding, extracted and repeated. It can then be subsequently reversed to obtain the original encoding. A symbol table is applied to this process.

$x \to$ VOO
$\tilde{x} \to$ HOO

Only one symbol is needed with the tilde operator. Begin by substituting the symbol and extract each node O.

VHVOOVOOHVOOVOO
VHxxHxx (substitute)
VHOOHOO,x,x,x,x (extract)
V$\tilde{x}\tilde{x}$,x,x,x,x (substitute)
VOO,$\tilde{x}$,$\tilde{x}$,x,x,x,x (extract)
x,$\tilde{x}$,$\tilde{x}$,x,x,x,x (end)

The commas can be removed from the ending result to obtain an efficient representation of the original string.

$x\tilde{x}\tilde{x}xxxx$

The opposite is possible. Begin with the first symbol and substitute its value.

x,$\tilde{x}$,$\tilde{x}$,x,x,x,x (add commas)
VOO,$\tilde{x}$,$\tilde{x}$,x,x,x,x (symbol)

V$\tilde{x}\tilde{x}$,x,x,x,x (insert)
VHOOHOO,x,x,x,x (symbol)
VHxxHxx (insert)
VHVOOVOOHVOOVOO (end)

1.8 CONTEXT-FREE GRAMMAR

The following is a context-free grammar for the XOO encoding:
S $\rightarrow$ O
S $\rightarrow x\tilde{o}$
$x \rightarrow$ H|V
$\tilde{x} \rightarrow x\tilde{o}$
$o \rightarrow$ O
$\tilde{o} \rightarrow oo|x\tilde{o}|o\tilde{x}$

Optional:

$\tilde{o} \rightarrow \tilde{x}\tilde{x}$

It can be shown using the pumping lemma that a p-length XOO encoding is non-regular. Given any value p a valid encoding xyz in the language $\mathcal{L}$ is one that satisfies these three properties:
$|y| >= 1$
$|xy| <= p$
$\forall n >= 0, xy^n z \in \mathcal{L}$
By pumping y, the encoding is no longer acceptable by the language $\mathcal{L}$.
Therefore the XOO encoding is a non-regular language.

Theorem 1. *The XOO-encoding is a non-regular language.*

Proof. The language $\mathcal{L}$ is the XOO encoding and assume that it is regular.
A valid XOO string is $X^n O^{n+1} \in \mathcal{L}\ \forall n >= 0$
However, for $n = 0$ the string O cannot be composed of xyz.
For $n = 1$, the string XOO is composable into xyz. This satisfies that $|y| >= 1$ and $|xy| <= p$.
Given a XOO encoding it is composed of $x =$ X, $y =$ O and $z =$ O, i.e. $n = 1$.
Let $n = 2$. Then immediately the resulting generated encoding is $XOOO$ which is not in $\mathcal{L}$.
Therefore, the XOO encoding is a non-regular language. □

1.9 PARALLEL COMPUTING

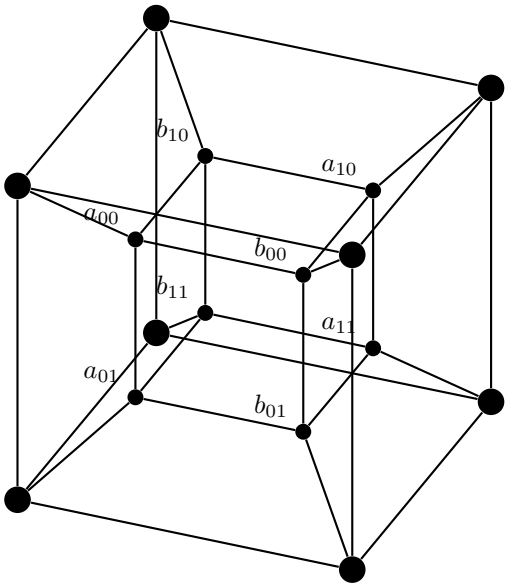
Parallel computing is a sophisticated area of computer science. It is a well established and mature technology.

"Graphs having the topology of a hypercube [are] ideally suited to modeling distributed-memory parallel computers, mainly because of the fact that the diameter of a hypercube is the logarithm of the number of vertices." [60]

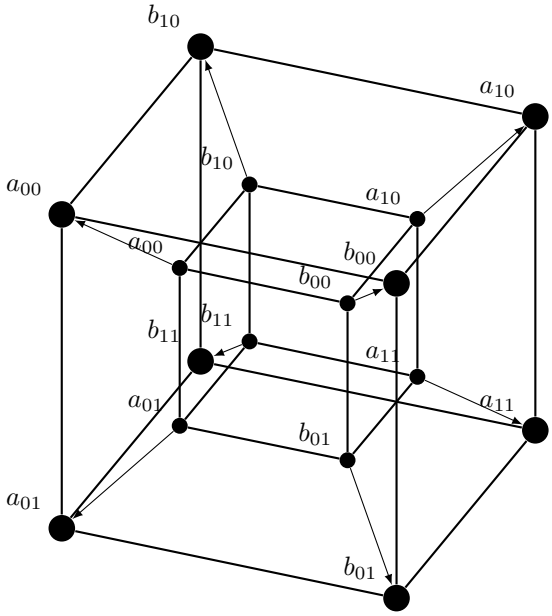
Hypercube algorithms exist to take advantage of multiple processing elements for many calculations at the same time. A simple way to illustrate this is through matrix multiplication.

$$AB = \begin{bmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{bmatrix} \begin{bmatrix} b_{00} & b_{01} \\ b_{10} & b_{11} \end{bmatrix} = \begin{bmatrix} a_{00}b_{00} + a_{01}b_{10} & a_{00}b_{01} + a_{01}b_{11} \\ a_{10}b_{00} + a_{11}b_{10} & a_{10}b_{01} + a_{11}b_{11} \end{bmatrix}$$

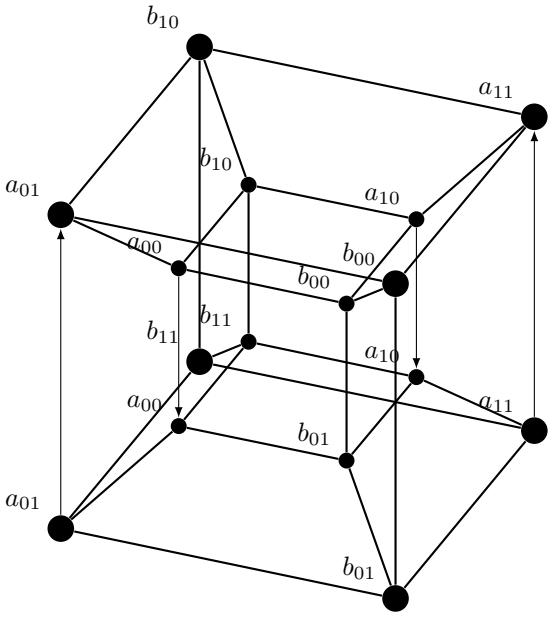
The process of matrix multiplication is done over multiple nodes - multiplication and addition are done through merging actions. The matrices are laid out in the XOO layout. Note that the hypercubes are simplified by one dimension in order to keep the illustration clearer, ie. the layout is 4D and the hypercube is 3D.



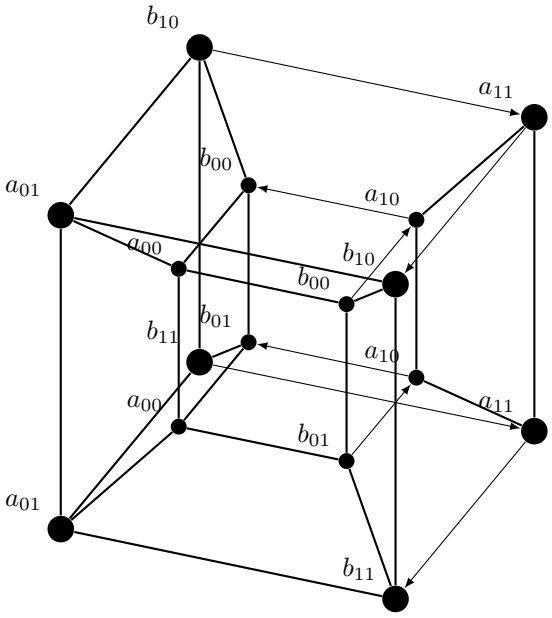
a_{00}	a_{01}		
—	—	—	—
b_{00}	b_{01}		
a_{10}	a_{11}		
—	—	—	—
b_{10}	b_{11}		



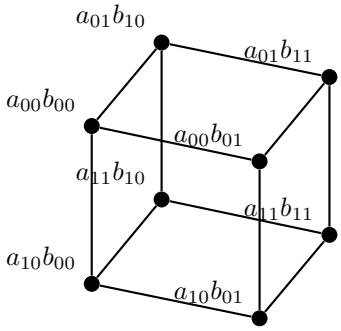
a_{00}	a_{01}	a_{01}	a_{00}
—	—	—	—
b_{00}	b_{01}	b_{01}	b_{00}
a_{10}	a_{11}	a_{11}	a_{10}
—	—	—	—
b_{10}	b_{11}	b_{11}	b_{10}

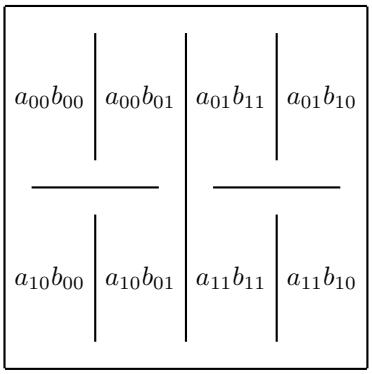


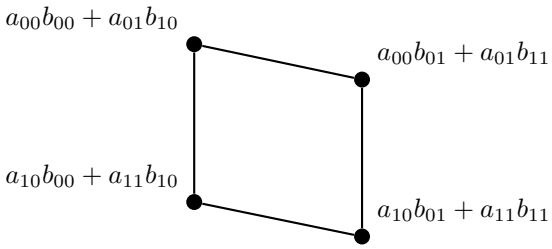
a_{00}	a_{00}	a_{01}	a_{01}
b_{00}	b_{01}	b_{01}	b_{00}
a_{10}	a_{10}	a_{11}	a_{11}
b_{10}	b_{11}	b_{11}	b_{10}

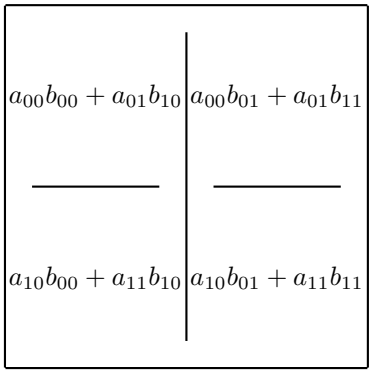


a_{00}	a_{00}	a_{01}	a_{01}
b_{00}	b_{01}	b_{11}	b_{10}
a_{10}	a_{10}	a_{11}	a_{11}
b_{00}	b_{01}	b_{11}	b_{10}





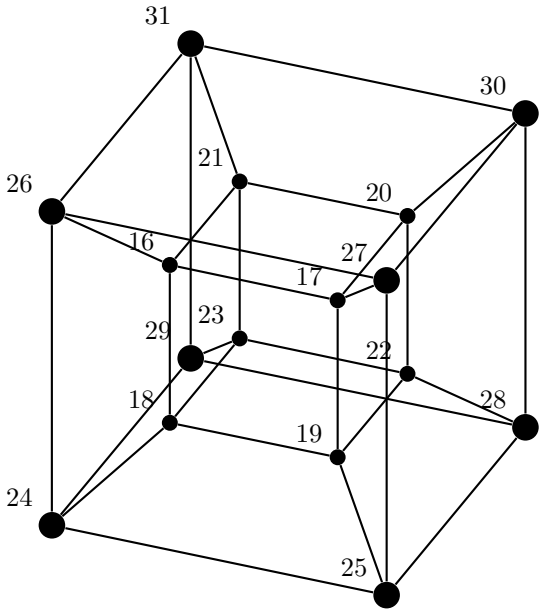




1.10 DISTRIBUTED SYSTEMS

There are two distributed systems that are of keen interest with respect to hypercube architecture and therefore the XOO layout schema. Content-addressable network (CAN) uses a layout scheme to illustrate how nodes of a network are distributed. Kademlia is a distributed network that uniquely uses XOR as a distance metric using binary trees. [10] [11]

There are advantages in using the XOO layout scheme as a basis for a distributed system. It can be shown that CAN and Kademlia share properties using an implementation of a XOO schema in a distributed system. The XOR metric in Kademlia is used as a distance metric whereas when implemented in XOO it is seen as a dimensional measure.



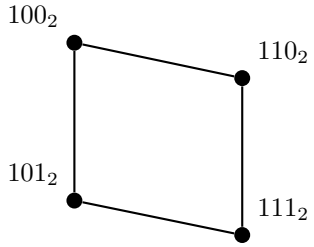
Nodes of a XOO layout in a distributed system represent clients/servers on a network. The XOO schema also has the advantage of determining which node stores/retrieves a hashed addressable content.

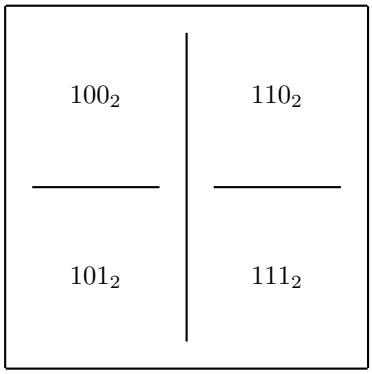
For example, given the string xoo that is SHA-1 hashed:

af633cd7b700e4832f05c2df6aaca651040acc6116

The binary value of the hash is:

$$101011110110...0001_2$$





Given a 0D network layout as the first case, there is only one node in the network that stores and retrieves all hashed content beginning with both 0_2 or 1_2 (0x0-0xF). Node 1_2 stores/retrieves the above hash.

Given a 1D dense layout, there are two nodes, 10_2 and 11_2. Node 10_2 captures all hashes beginning with 0_2 (or 0x0-0x7). Node 11_2 captures all hashes beginning with 1_2 (or 0x8-0xF) and stores/retrieves the above hash..

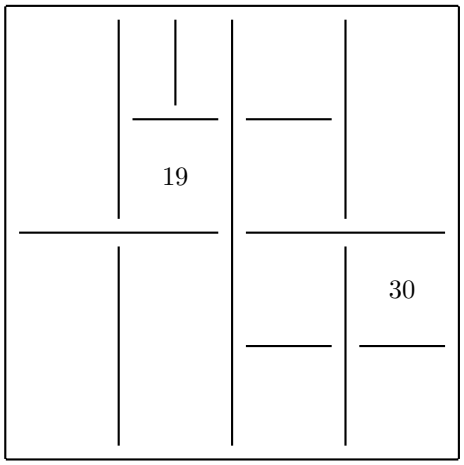
Given a 2D dense layout, there are four nodes: 100_2, 101_2, 110_2 and 111_2. 100_2 stores and retrieves hashes with values beginning with 00_2 (0x0-0x3), 101_2 stores/retrieves hashes with values beginning with 01_2 (0x4-0x7), and so on. Node 110_2 stores/retrieves the above hash.

The general rule is: given the node ID, the first bit of the node ID is ignored and the rest of the bits store and retrieve content addressable hashes matching the first set of bits of the hash. Given a hash, a network node determines whether it stores/retrieves the hashed content or passes the request along to a neighbourly node.

Now that it is known which node stores which hash, it is a matter of traversing the network from some node in the network to the destination node. This is done with a layout representation where each subdivision represents a node and each node knows only itself and its immediate neighbours.

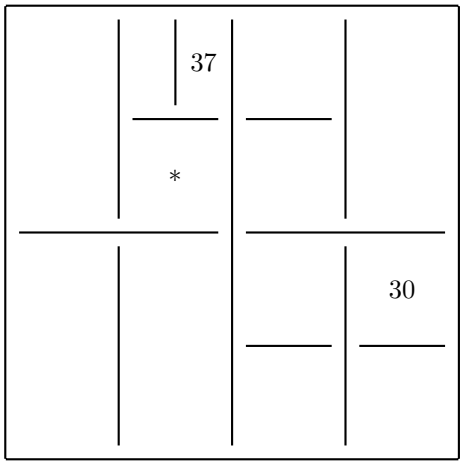
Node 19_{10} is queried for a hash beginning with 0xF8. The node calculates that it does not store those series of hashes, and calculates that none of its neighbours do, either.

Node 19_{10} then chooses some neighbour to further query.



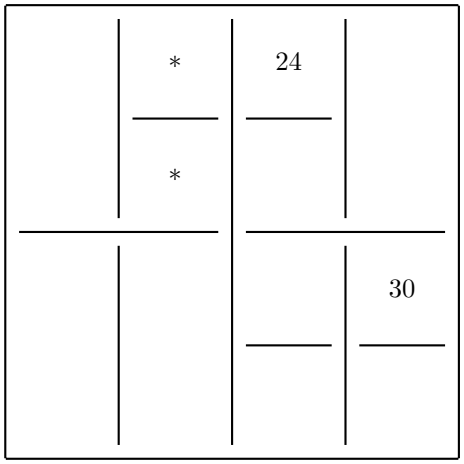
Node 37_{10} has been chosen and node 19_{10} has been flagged as the previous node. Node 37_{10} does not store the hash nor do its neighbours so one of its neighbours is chosen as the next node to query.

Node 19_{10} is not a candidate node since it was the previous node.

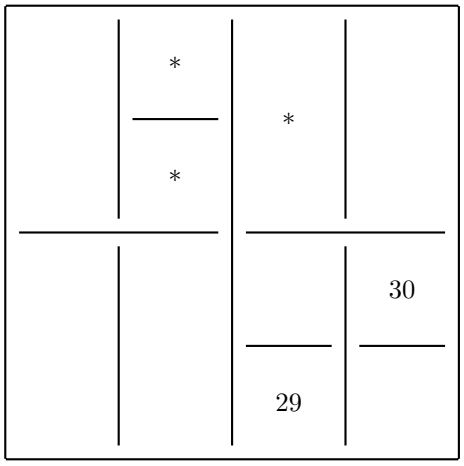


Node 24_{10} is the next node that has been chosen. It does not store the hash.

Since 37_{10} was the previous node, it is not flagged but instead removed from the mapping.* Node 24_{10} queries one of its neighbours in the mapping. Note that node 37_{10} is removed from the mapping which has been replaced by 18_{10} and 18_{10} is not a neighbour of 24_{10} in the original network layout.

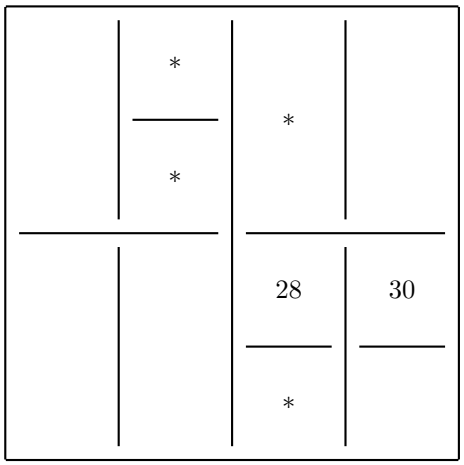


Node 29_{10} is chosen and it also does not store the hash nor do any of its neighbours. A neighbour is chosen randomly for the new query.



Node 28_{10} has been chosen. Node 29_{10} is flagged as a previous node. Node 30_{10} is identified as being a store for the hash.

Node 30_{10} is queried for the hash and the node returns its store.



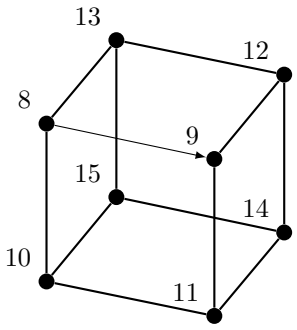
* A previous node is removed from the layout mapping when the new node chosen is not found transversally down the binary tree and is not its sibling.

1.11 SNAKE-IN-THE-BOX PROBLEM

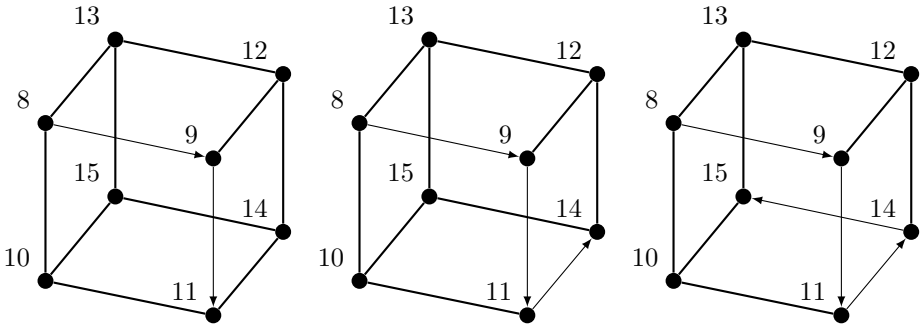
The snake-in-the-box problem is one in which to find the longest path in a hypercube where the path complies with certain attributes. The path traverses nodes where the beginning and ending nodes have only one neighbour on the path otherwise every node on the path has exactly two neighbours on the path.

The SIB problem is considered difficult as the hypercube increases in dimension. The number of possible choices that eliminate future possible paths becomes intractable while trying to maximize the path length through the hypercube. [84]

Its now proposed that an alternating-oriented bisected layout can be used to solve this problem. First, the problem is illustrated using a three-dimensional cube.

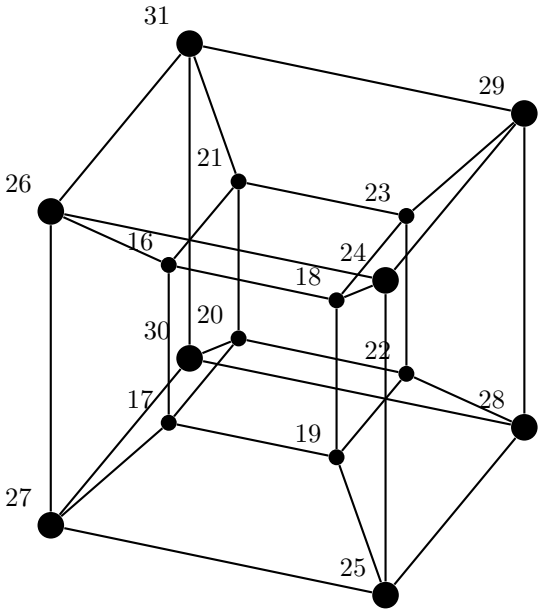


Starting a node 8, choose any node to traverse to, so node 9 is chosen. Eliminate (grey out) the other neighbour nodes of 8 since they cannot be chosen by any future path. Continue the process from node 9 to node 11.

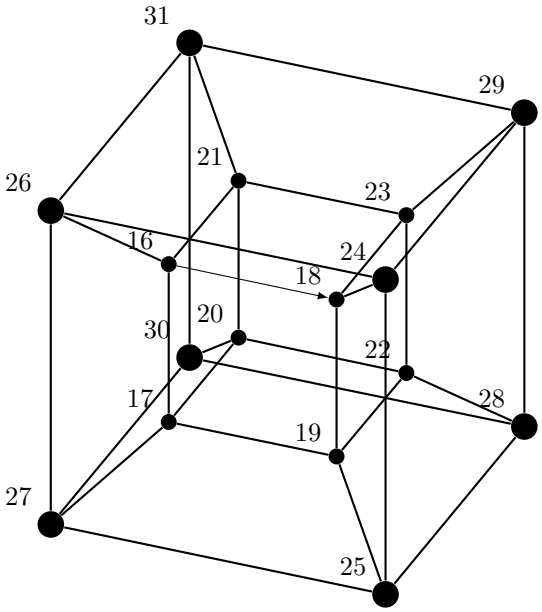


The maximal path length of a three-dimensional cube is 4 edges.

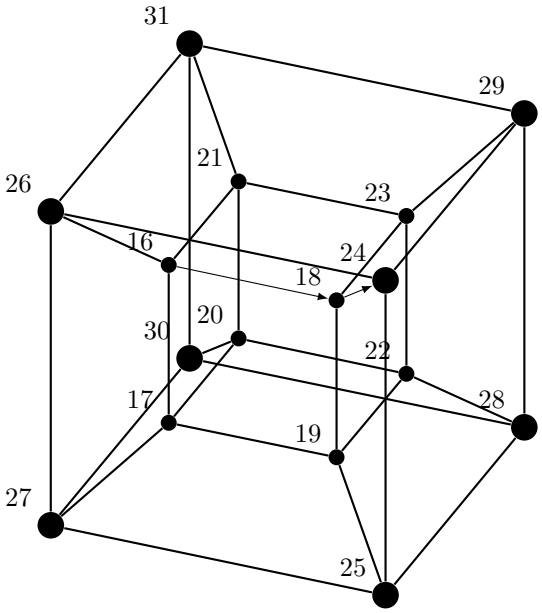
A four-dimensional hypercube maximal path will be illustrated with the benefit of an alternating-oriented bisected layout. In addition to illustrating the path on the hypercube, the layout will show the path from node-to-node and eliminated nodes are represented with an x.

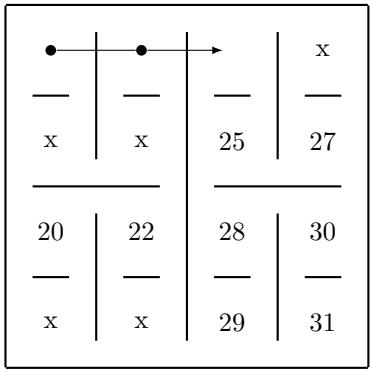


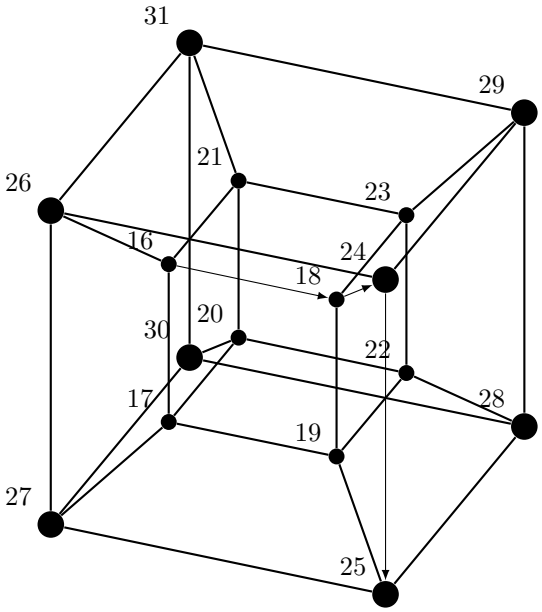
16	18	24	26
17	19	25	27
20	22	28	30
21	23	29	31

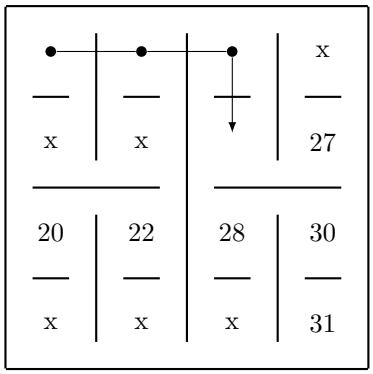


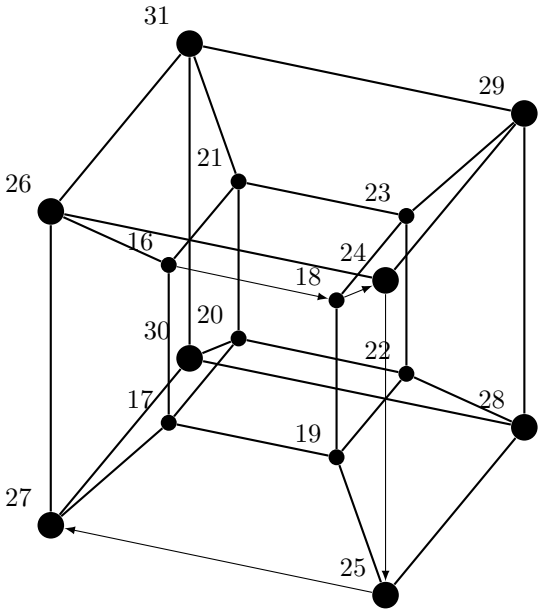
		24	x
x	19	25	27
20	22	28	30
x	23	29	31

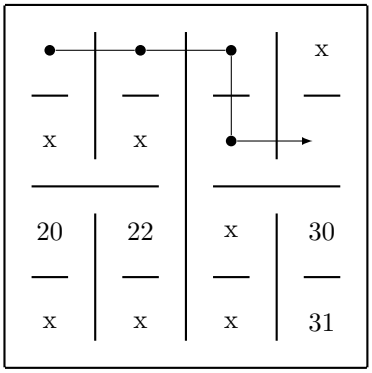

31
29
26
21
23
16
24
18
20
30
22
17
28
19
27
25

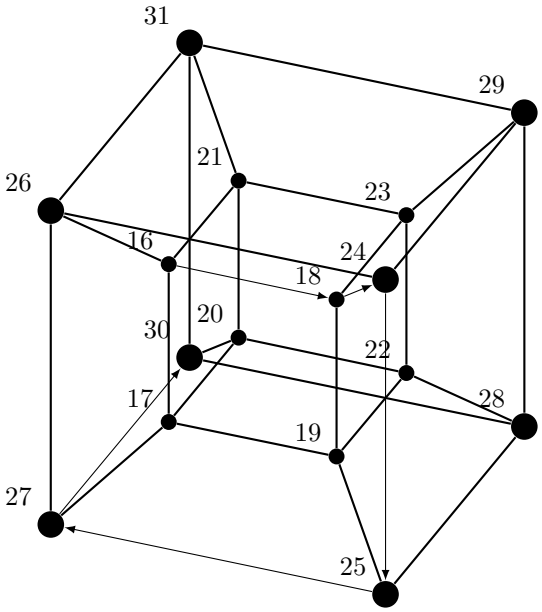

x
x
x
25
27
20
22
28
30
x
x
29
31

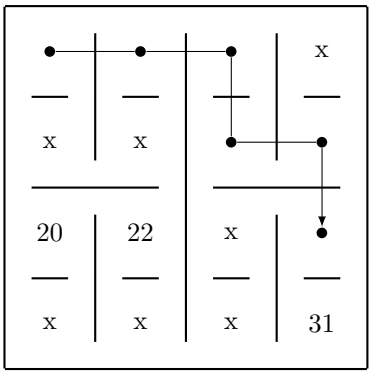

31
29
26
21
23
16
24
18
20
30
22
17
28
19
27
25

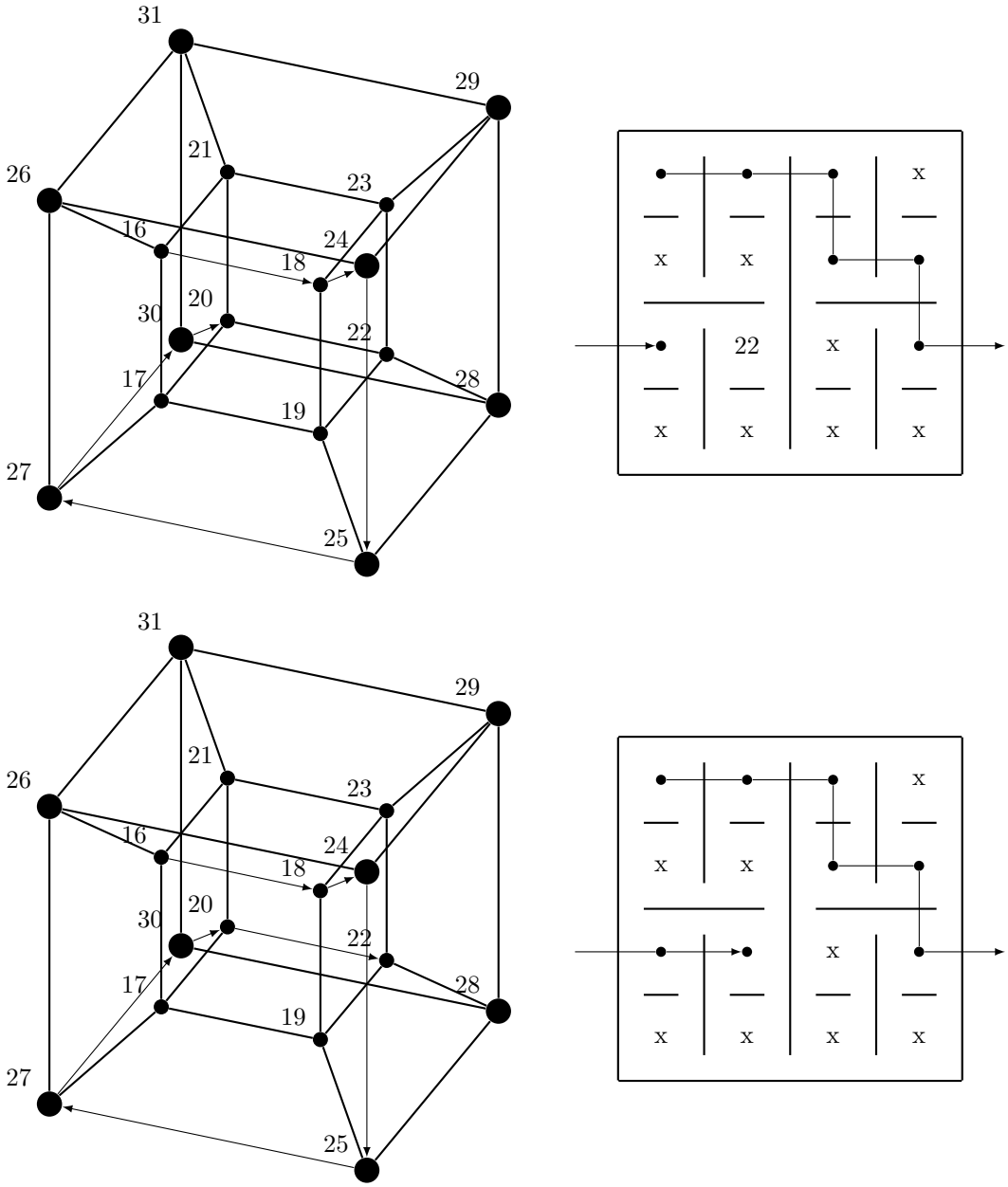

x
x
x
27
20
22
28
30
x
x
x
31


31
29
21
26
23
16
24
18
20
30
22
17
28
19
27
25


x
x
x
20
22
x
30
x
x
x
31


31
29
21
26
23
16
24
18
20
30
22
17
28
19
27
25


x
x
x
20
22
x
x
x
x
31



The 4-dimensional hypercube has a maximal path of 7 edges. The value of the alternating-oriented bisected layout illustrates a useful data structure for problem-solving.

In addition, the layout is labelled "ABCD". The above particular maximal path can be algebraically represented as:

$$\overline{AD} + A\overline{BC} + A\overline{B}D + BC\overline{D}$$

With the use of the alternating-oriented bisected layout the problem space collapses. The following theorems of maximal paths over hypercubes were deduced.

Theorem 2. *The maximal path for hypercubes is:*
Even dimensions > 2:

$$e = (2^{\frac{d-2}{2}} - 1)(2^{\frac{d}{2}} + 3)$$

Odd dimensions > 3:

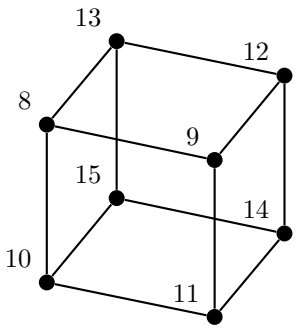
$$e = (2^{\frac{d-1}{2}} - 1)(2^{\frac{d-1}{2}} + 2)$$

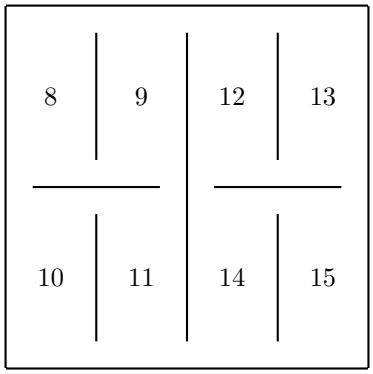
Theorem 3. *The maximal path on hypercube dimension d>2 is:*

$$e = (2^{\frac{d-(d-1) \bmod 2-1}{2}} - 1)(2^{\frac{d-d \bmod 2}{2}} + (d-1) \bmod 2 + 2)$$

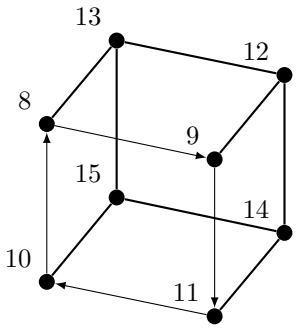
1.12 PERMUTATIONS

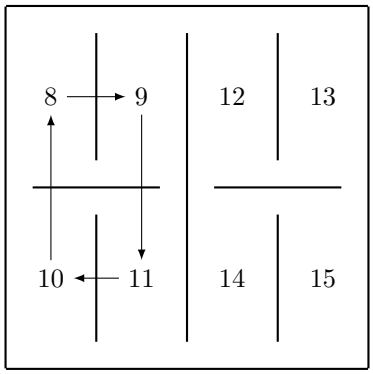
The three dimensional cube and the XOO layout can be theorized as a simplified Rubik's cube where each face of the cube can be rotated by permuting the labels of a face.

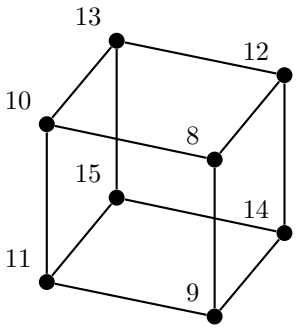


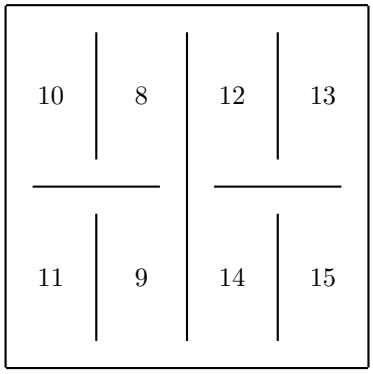


The above layout is the starting orientation. The following results in the 8-9-11-10 (a four-cycle) face being "rotated" relative to the starting orientation. There are six possible explicit single four-cycle rotations.

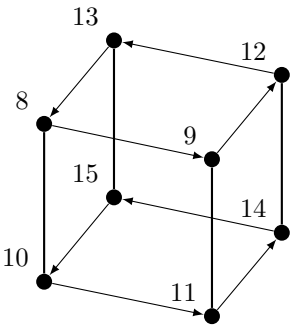


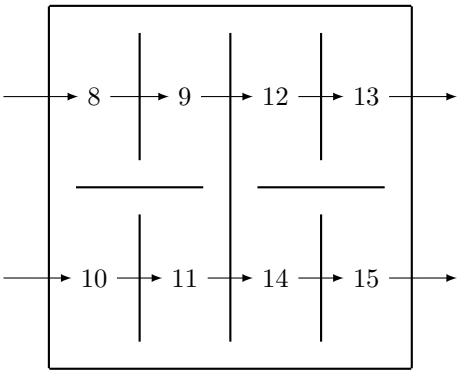


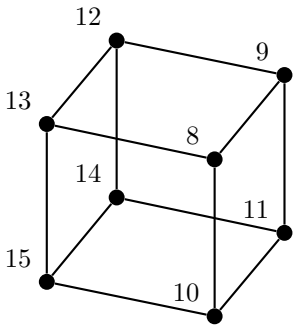


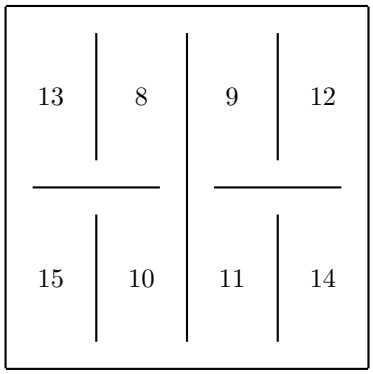


There are three possible implicit double four-cycle rotations as if the middle plane of a Rubik's cube is implied to rotate in the counter-direction.





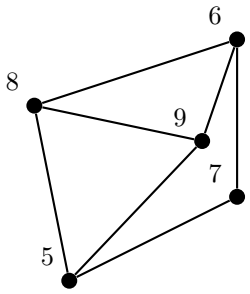




1.13 GRAPH OPERATIONS

Graph theory, the mathematics of vertex-edge sets, can be used to study operations on hypercube graphs. Matrix operations on matrices that represent graphs can add further room to study hypercubes and the XOO layout.

Since the XOO layout is used as a basis for hypercube graphs, the vertices are labelled according to the previous node labelling. Any operations on a graph that affect vertices also affect the labelling.

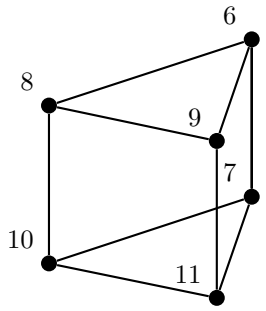


	8	9	5	6	7
8	0	1	1	1	0
9	1	0	1	1	0
5	1	1	0	0	1
6	1	1	0	0	1
7	0	0	1	1	0

$$\mathrm{R_0GC_0} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R_1GC_1 = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

	8	9	10	11	6	7
8	0	1	1	0	1	0
9	1	0	0	1	1	0
10	1	0	0	1	0	1
11	0	1	1	0	0	1
6	1	1	0	0	0	1
7	0	0	1	1	1	0



1.14 PROGRAMMING

An area of research still to be done is to develop a programming language based on hypercubes. As can be seen above, there is much interesting research and development to be done in different fields of computer science and applications development with respect to the XOO schema. This paper provides the basis for new areas of research and development.

A hypercube programming language and a graph API can be developed as operations on the graph abstract data type [82] [83]:

adjacent(G, x, y): tests whether there is an edge from the vertices x to y;
neighbors(G, x): lists all vertices y such that there is an edge from the vertices x to y;
add_vertex(G, x): adds the vertex x, if it is not there;
remove_vertex(G, x): removes the vertex x, if it is there;
add_edge(G, x, y): adds the edge from the vertices x to y, if it is not there;
remove_edge(G, x, y): removes the edge from the vertices x to y, if it is there;
get_vertex_value(G, x): returns the value associated with the vertex x;
set_vertex_value(G, x, v): sets the value associated with the vertex x to v.
get_edge_value(G, x, y): returns the value associated with the edge (x, y);
set_edge_value(G, x, y, v): sets the value associated with the edge (x, y) to v.
In addition, a distributed network can provide an API to store/retrieve content:

ping: used to verify that a node is alive;

store: stores a (key, value) pair in one node;

find_node: the recipient of the request will return the k nodes closest to the requested key;

find_value: same as find_node but if the recipient of the request has the requested key in its store it will return the corresponding value;

A web application is being built using "a method for users to create and edit web page layouts" where every division (node) of a layout represents a network resource or process. The properties of the application demonstrate that the XOO layout schema is a viable technology on which to build efficient hypercube web applications and systems. [85]

A line of application development is a XOO layout API. Beginning with a text-based terminal, it would be trivial to display fixed-width text layouts and implement a simple text-based windowing system. Building on that, windows in a desktop environment can be laid out according to the XOO layout schema. Both textual and window environments would implement aspects of the XOO schema. This way, every division of a layout (ie. a node of a hypercube) can represent a network resource (distributed hypercube) or a process (parallel computing process) as mentioned in the above web application. The way data is presented, networked, processed and laid out would all be aspects of the XOO layout schema and, by extension, hypercube architecture.

Finally, the mathematics of hypercubes are well established but with the realization of the interchangeable data structures shown above there are real-world applications of mathematical theory throughout the XOO data structure. Putting the XOO data structure on a firm foundation will be done via a mathematical and computational survey. This paper illustrates the wide areas of symmetry to be pursued in research and development of the XOO schema and hypercubes.

1.15 TURING MACHINE

The following program verifies that a XOO encoding is valid.

$M = (\Sigma, \Gamma, Q, \delta, q, q_a, q_r)$

$\Sigma = (H, V, O)$

$\Gamma = (\square, X, H, V, O)$

$Q = (q_{start}, q_0, q_{init}, q_x, q'_x, q_{merge}, q_{shift}, q'_{shift}, q_{shift_H}, q_{shift_V}, q_{shift_O}, q_{print_H}, q_{print_V}, q_{print_O}, q_{return}, q_{accept}, q_{reject})$

$$\delta = \begin{cases}
q_{start}O \to q_0OR \\
q_{start}H \to q_{init}HL \\
q_{start}V \to q_{init}VL \\
q_0\square \to q_{accept} \\
q_0H \to q_{reject} \\
q_0V \to q_{reject} \\
q_0O \to q_{reject} \\
q_{init}\square \to q_xXR \\
q_xH \to q_xHR \\
q_xV \to q_xVR \\
q_xO \to q'_xOR \\
q_x\square \to q_{reject} \\
q'_xH \to q_xHR \\
q'_xV \to q_xVR \\
q'_xO \to q_{merge}XL \\
q'_x\square \to q_{reject} \\
q_{merge}H \to q_{shift}OR \\
q_{merge}V \to q_{shift}OR \\
q_{merge}O \to q_{merge}\square L \\
q_{shift}\square \to q_{shift}\square R \\
q_{shift}X \to q'_{shift}\square R \\
q'_{shift}\square \to q_{return}\square L \\
q'_{shift}H \to q_{shift_H}XL
\end{cases}
\qquad
\begin{cases}
q'_{shift}V \to q_{shift_V}XL \\
q'_{shift}O \to q_{shift_O}XL \\
q_{shift_H}\square \to q_{shift_H}\square L \\
q_{shift_H}H \to q_{print_H}HR \\
q_{shift_H}V \to q_{print_H}VR \\
q_{shift_H}O \to q_{print_H}OR \\
q_{shift_V}\square \to q_{shift_V}\square L \\
q_{shift_V}H \to q_{print_V}HR \\
q_{shift_V}V \to q_{print_V}VR \\
q_{shift_V}O \to q_{print_V}OR \\
q_{shift_O}\square \to q_{shift_O}\square L \\
q_{shift_O}H \to q_{print_O}HR \\
q_{shift_O}V \to q_{print_O}VR \\
q_{shift_O}O \to q_{print_O}OR \\
q_{print_H}\square \to q_{shift}HR \\
q_{print_V}\square \to q_{shift}VR \\
q_{print_O}\square \to q_{shift}OR \\
q_{return}\square \to q_{return}\square L \\
q_{return}H \to q_xHN \\
q_{return}V \to q_xVN \\
q_{return}O \to q_{return}OL \\
q_{return}X \to q_{start}\square R
\end{cases}$$

$q = q_{start}$

$q_a = q_{accept}$

$q_r = q_{reject}$

1.16 TERNARY LOGIC TABLE

-	0	+	x	UNARY OPERATOR
-	-	-	$\tilde{\bar{x}} + \lvert\tilde{x}\rvert$	
-	-	0	$\tilde{\bar{x}} + \lfloor x \rfloor$	
-	-	+	$\tilde{\bar{x}} + [x]$	Is True
-	0	-	$\lvert\tilde{x}\rvert$	
-	0	0	$\lfloor x \rfloor$	* Clamp Down
-	0	+	$[x]$	* Buffer
-	+	-	$\bar{x} + \lvert\tilde{x}\rvert$	Is Unknown
-	+	0	$\bar{x} + \lfloor x \rfloor$	
-	+	+	$\bar{x} + [x]$	
0	-	-	$\tilde{\bar{x}} + \lceil\tilde{x}\rceil$	
0	-	0	$\tilde{\bar{x}}$	
0	-	+	$\tilde{\bar{x}} + \lceil x \rceil$	
0	0	-	$\lceil\tilde{x}\rceil$	
0	0	0		Don't Care
0	0	+	$\lceil x \rceil$	* Clamp Up
0	+	-	$\bar{x} + \lceil\tilde{x}\rceil$	Increment
0	+	0	$\bar{x}$	*
0	+	+	$\bar{x} + \lceil x \rceil$	
+	-	-	$\tilde{\bar{x}} + [\tilde{x}]$	Is False
+	-	0	$\tilde{\bar{x}} + \lfloor\tilde{x}\rfloor$	Decrement
+	-	+	$\tilde{\bar{x}} + \lvert x \rvert$	
+	0	-	$[\tilde{x}]$	Not
+	0	0	$\lfloor\tilde{x}\rfloor$	
+	0	+	$\lvert x \rvert$	*
+	+	-	$\bar{x} + [\tilde{x}]$	
+	+	0	$\bar{x} + \lfloor\tilde{x}\rfloor$	
+	+	+	$\bar{x} + \lvert x \rvert$	

* Defined ternary operators [34]

1.17 KEYWORDS

- Actor Model
- Algebra
  - Terwilliger Algebra
- Algorithm
- Architecture
- Byzantine Generals Problem
- Computing
  - Distributed
  - Parallel
- Cryptography
- Cubical Calculus
  - Cube Notation
- Elections
- Finite State Machine
- Fuzzy (Logic)
  - Fuzzy Cube
- Genetics
  - Hypercube polynucleotide
- Grammar-based Codes
- Graph
  - Abstract Data Type
- Group
  - Hyperoctahedral Group
- Index (Search) Trees
  - K-D-B-tree (k-dimensional b-tree)
  - Bkd-tree
- Latin Hypercube Sampling
- Layout

  - Circuit
  - Floorplan
  - VLSI (Very Large Scale Integration)
- Linguistics
  - Geometrical Morphology
- Measure Polytope Hypercube Sequence
- MIMD (Multiple Instruction Multiple Data)
- Nanocomputer
  - M-Hypercube
- Network
  - Decentralized
  - Distributed
  - Interconnection
- OLAP (Online Analytical Processing)
  - Hypertable
- P2P (Peer-to-peer)
  - CAN (Content-Addressable Network)
  - Kademlia
- Plan 9
- Routing
- Semantic Web
- Snake-in-the-Box
- Space-Partitioning Trees
  - 2D Tree
  - K-d Tree
  - K-D-B Tree
- Switching Theory
- Ternary
- Tesseract
- Topology
- Voxel

References

1. Tucovic, A. Optimizing Data Structures with Alternating-Oriented Bisected Layouts. For Violeta Boskovic and Rea Tucovic (2018).
2. Robinson, G. *Birth of the Hypercube* http://www.netlib.org/utk/lsi/pcwLSI/text/node13.html. Mar. 1995.
3. Foster, I. *Hypercube Algorithms* http://www.mcs.anl.gov/~itf/dbpp/text/node123.html. 1995.
4. Schlosser, M., Sintek, M., Decker, S. & Nejdl, W. *HyperCuP — Hypercubes, Ontologies, and Efficient Search on Peer-to-Peer Networks* in *Agents and Peer-to-Peer Computing* (eds Moro, G. & Koubarakis, M.) (Springer Berlin Heidelberg, Berlin, Heidelberg, 2003), 112–124. ISBN: 978-3-540-45074-0.
5. Jiménez-Montaño, M. A., de la Mora-Basáñez, C. R. & Pöschel, T. *On the Hypercube Structure of the Genetic Code* in *Proc. 3. Int. Conf. on Bioinformatics and Genome Research* (eds Lim, H. A. & Cantor, C. A.) https://arxiv.org/abs/cond-mat/0204044 (1994), 445.
6. Buterin, V. *Scalability, Part 2: Hypercubes* https://blog.ethereum.org/2014/10/21/scalability-part-2-hypercubes/. 2014.
7. Knuth, D. E. *FINITE SEMIFIELDS AND PROJECTIVE PLANES* https://thesis.library.caltech.edu/2441/1/Knuth_de_1963.pdf. PhD thesis (California Institute of Technology, 1963).
8. Spadavecchia, L. *A Network-based Asynchronous Architecture for Cryptographic Devices* https://www.era.lib.ed.ac.uk/bitstream/handle/1842/860/?sequence=1. PhD thesis (University of Edinburgh, 2005).
9. Savage, J. E. in (ed Shanklin, C.) http://cs.brown.edu/people/jsavage/book/pdfs/ModelsOfComputation_Chapter7.pdf, 298–309 (Addison Wesley Longman, July 2008).
10. Ratnasamy, S., Francis, P., Handley, M., Karp, R. & Shenker, S. *A Scalable Content-Addressable Network* in *ACM SIGCOMM* http://conferences.sigcomm.org/sigcomm/2001/p13-ratnasamy.pdf (2001).
11. Maymounkov, P. & Mazières, D. *Kademlia: A Peer-to-Peer Information System Based on the XOR Metric* in *Peer-to-Peer Systems* (eds Druschel, P., Kaashoek, F. & Rowstron, A.) (Springer Berlin Heidelberg, Berlin, Heidelberg, 2002), 53–65. ISBN: 978-3-540-45748-0.
12. Adler, M., Halperin, E., Karp, R. M. & Vazirani, V. V. *A Stochastic Process on the Hypercube with Applications to Peer-to-peer Networks* in *Proceedings of the Thirty-fifth Annual ACM Symposium on Theory of Computing* http://www.icsi.berkeley.edu/pubs/networking/stochasticprocess03.pdf (ACM, San Diego, CA, USA, 2003), 575–584. ISBN: 1-58113-674-9. doi:10.1145/780542.780626. http://doi.acm.org/10.1145/780542.780626.
13. Giraud-Carrier, C. A Reconfigurable Dataflow Machine for Implementing Functional Programming Languages. *SIGPLAN Not.* **29.** http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.60.3816&rep=rep1&type=pdf, 22–28. ISSN: 0362-1340 (Sept. 1994).
14. Villa, T. & Sangiovanni-Vincentelli, A. NOVA: State Assignment of Finite State Machines for Optimal Two-Level Logic Implementation. **9.** https://people.eecs.berkeley.edu/~alanmi/publications/other/nova.pdf, 905 –924 (Oct. 1990).

15. S. Liu, M. P. & Despain, A. M. State Assignment based on Two–dimensional Placement and Hypercube Mapping. *Integration, the VLSI Journal.* http://www.mpedram.com/Papers/quadassign-journal.pdf, 101–118 (1997).

16. Lee, C., Baek, S. H. & Park, K. H. A Hybrid Flash File System Based on NOR and NAND Flash Memories for Embedded Devices. *IEEE Trans. Comput.* **57,** 1002–1008. ISSN: 0018-9340 (July 2008).

17. Alvarez-Hamelin, J. I., Viana, A. C. & Amorim, M. D. D. *DHT-based Functionalities Using Hypercubes* in *In: Proceedings of World Computer Congress IFIP WCC. Volume* http://sedici.unlp.edu.ar/bitstream/handle/10915/23832/Article_12\discretionary{-}{}{}Alvarez\discretionary{-}{}{}Hamelin.pdf?sequence=1 (2006), 212.

18. Jiménez-Montaño, M. A., de la Mora-Basáñez, C. R. & Pöschel, T. The Hypercube Structure of the Genetic Code Explains Conservative and Non-Conservative Amino Acid Substitutions in vivo and in vitro. **39.** https://arxiv.org/pdf/cond-mat/0204078.pdf, 117–125 (1996).

19. Sant'Ana da Silva, E. & Pedrini, H. Inferring patterns in mitochondrial DNA sequences through hypercube independent spanning trees. *Computers in Biology and Medicine* **70,** 51 –57. ISSN: 0010-4825 (2016).

20. Ajorloo, H. & Lakdashti, A. HBIR: Hypercube–based image retrieval. *Journal of Computer Science and Technology* **27,** 147–162. ISSN: 0010-4825 (2012).

21. Karakowski, J. A. & Phu, H. H. *A Fuzzy Hypercube Artificial Neural Network Classifier* tech. rep. http://www.dtic.mil/dtic/tr/fulltext/u2/a354805.pdf (U.S. Army Communications-Electronics Command, 1998).

22. Baldi, P. Boolean autoencoders and hypercube clustering complexity. doi:DOI10.1007/s10623-012-9719-x. \url{https://link.springer.com/content/pdf/10.1007%2Fs10623-012-9719-x.pdf} (2012).

23. Chen, Y.-W. & Lu, M.-C. Design a Hypercube–Tree Structure for Peer-to-Peer Streaming. *International Journal of Computer and Communication Engineering* **1.** \url{http://www.ijcce.org/papers/23-C016.pdf} (2012).

24. Lu, G. A Class of New Metrics for n-Dimensional Unit Hypercube. *Journal of Applied Mathematics.* doi:http://dx.doi.org/10.1155/2013/942687. \url{http://downloads.hindawi.com/journals/jam/2013/942687.pdf} (2013).

25. Torres-Iglesias, A. & Nieto, J. The fuzzy polynucleotide space: Basic properties. **19,** 587–92 (Apr. 2003).

26. Sadegh-Zadeh, K. The fuzzy polynucleotide space revisited. **41,** 69–80; discussion 81 (Oct. 2007).

27. Chang, Y.-C., Chang, Y.-W., Wu, G.-M. & Wu, S.-W. *B*-trees: a new representation for non-slicing floorplans* in *Proceedings 37th Design Automation Conference* (June 2000), 458–463. doi:10.1109/DAC.2000.855354.

28. Kang, H. & Vachtsevanos, G. *Fuzzy hypercubes: linguistic learning/reasoning systems for intelligent control and identification* in *[1991] Proceedings of the 30th IEEE Conference on Decision and Control* (Dec. 1991), 1200–1205 vol.2. doi:10.1109/CDC.1991.261559.

29. Ahmad, I. HARD: A hypercube embedding algorithm for state assignment of finite state machines. *Computers and Electrical Engineering* **29,** 327 –356. ISSN: 0045-7906 (2003).

30. Hook, L. R. & Lee, S. C. *The M-Hypercube* in *2006 Canadian Conference on Electrical and Computer Engineering* (May 2006), 70–73. doi:10.1109/CCECE.2006.277851.

31. Lee, S. C. & Hook, L. R. *Towards the Design of a Nanocomputer* in *2006 Canadian Conference on Electrical and Computer Engineering* (May 2006), 74–77. doi:10.1109/CCECE.2006.277855.

32. Ranka, S. & Sahni, S. *Hypercube Algorithms: with Applications to Image Processing and Pattern Recognition* (Springer-Verlag, 1990).

33. Kim, W. *et al.* Multistate Memristive Tantalum Oxide Devices for Ternary Arithmetic. *Scientific Reports.* doi:10.1038/srep36652. \url{https://www.nature.com/articles/srep36652.pdf} (2016).

34. Jones, D. W. *Standard Ternary Logic* http://homepage.cs.uiowa.edu/~jones/ternary/logic.shtml. 2016.

35. Miller, D. & Thornton, M. *Multiple-Valued Logic: Concepts and Representations* ISBN: 9781598291919. https://books.google.ca/books?id=iZZdAQAAQBAJ (Morgan & Claypool Publishers, 2007).

36. Perkowski, M., Foote, D., Chen, Q., Al-Rabadi, A. & Jozwiak, L. Learning Hardware Using Multiple-Valued Logic, Part 1: Introduction and Approach. *IEEE Micro* **22,** 41–51. ISSN: 0272-1732 (May 2002).

37. Perkowski, M., Foote, D., Chen, Q., Al-Rabadi, A. & Jozwiak, L. Learning Hardware Using Multiple-Valued Logic, Part 2: Cube Calculus and Architecture. *IEEE Micro* **22,** 52–61. ISSN: 0272-1732 (May 2002).

38. Rickard, J. T. & Yager, R. R. Hypercube Graph Representations and Fuzzy Measures of Graph Properties. *IEEE Transactions on Fuzzy Systems* **15,** 1278–1293. ISSN: 1063-6706 (Dec. 2007).

39. Demaine, E. D., Eisenstat, S. & Rudoy, M. *Solving the Rubik's Cube Optimally is NP-complete* in *35th Symposium on Theoretical Aspects of Computer Science (STACS 2018)* (eds Niedermeier, R. & Vallée, B.) **96** (Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, Dagstuhl, Germany, 2018), 24:1–24:13. ISBN: 978-3-95977-062-0. doi:10.4230/LIPIcs.STACS.2018.24. http://drops.dagstuhl.de/opus/volltexte/2018/8533.

40. Wolf, R. A Brief Introduction to Fourier Analysis on the Boolean Cube. **1,** 1–20 (Jan. 2008).

41. Al-Omari, M. & Mahafzah, M. *Fault-tolerant Routing in Hypercubes Using Masked Interval Routing Scheme* in *Proceedings of the 1999 ACM Symposium on Applied Computing* (ACM, San Antonio, Texas, USA, 1999), 481–485. ISBN: 1-58113-086-4. doi:10.1145/298151.298424. http://doi.acm.org/10.1145/298151.298424.

42. Valenzuela, C. L. & Wang, P. Y. *A Genetic Algorithm for VLSI Floorplanning* in *Proceedings of the 6th International Conference on Parallel Problem Solving from Nature* (Springer-Verlag, London, UK, UK, 2000), 671–680. ISBN: 3-540-41056-2. http://dl.acm.org/citation.cfm?id=645825.668940.

43. Youyao, L., Jungang, H. & Huimin, D. A Hypercube-based Scalable Interconnection Network for Massively Parallel Computing. *Journal of Computers* (2008).

44. Yu, C., Das, C. R., Mohapatra, P. & Mohapatra, P. *A Lazy Scheduling Scheme for Hypercube Computers* 1995.

45. Sakamoto, M. & Morisue, M. *A study of ternary fuzzy processor using neural networks* in *Proceedings of 1997 IEEE International Symposium on Circuits and Systems. Circuits and Systems in the Information Age ISCAS '97* **1** (June 1997), 613–616 vol.1. doi:10.1109/ISCAS.1997.608880.

46. Alipour, P. A Universal 4D Model for Double-Efficient Lossless Data Compressions (Oct. 2011).

47. Clapp, R. M. & Mudge, T. *Ada on Hypercube* in *Proceedings of the Third Conference on Hypercube Concurrent Computers and Applications: Architecture, Software, Computer Systems, and General Issues - Volume 1* (ACM, Pasadena, California, USA, 1988), 399–408. ISBN: 0-89791-278-0. doi:10.1145/62297.62343. http://doi.acm.org/10.1145/62297.62343.

48. Kahramanli, Ş. Ş. & Allahverdı, N. Algebraic Approach to Transformations on Hypercube System. **1** (June 1996).

49. Parker, A. in. Chap. 2.5.3.3,2.6 (CRC Press, 1993).

50. Aderhold, M. & Slack, J. Embedding balanced binary trees in the hypercube (Jan. 2003).

51. Borba Ribeiro, H. *Exploiting Rateless Coding in Structured Overlays to Achieve Persistent Storage* Theses (Université Rennes 1, Oct. 2012). \url{https://tel.archives-ouvertes.fr/tel-00763284}.

52. Jayaraman, R. & Rutenbar, R. A. *FLOORPLANNING BY ANNEALING ON A HYPERCUBE MULTIPROCESSOR.* English (US). in *Unknown Host Publication Title* (IEEE, 1987), 346–349. ISBN: 0818608145.

53. in (eds Blazewicz, J., Ecker, K., Plateau, B. & Trystram, D.) chap. 3, 4 (Springer, 2000).

54. Ding, X., Liu, J. & Shao, D. *Hypercube-Based Pairwise Key Establishment for Wireless Sensor Networks* in *2008 4th International Conference on Wireless Communications, Networking and Mobile Computing* (Oct. 2008), 1–4. doi:10.1109/WiCom.2008.1108.

55. Sundar, H., Malhotra, D. & Biros, G. *HykSort: A New Variant of Hypercube Quicksort on Distributed Memory Architectures* in *Proceedings of the 27th International ACM Conference on International Conference on Supercomputing* (ACM, Eugene, Oregon, USA, 2013), 293–302. ISBN: 978-1-4503-2130-3. doi:10.1145/2464996.2465442. http://doi.acm.org/10.1145/2464996.2465442.

56. Morineau, T. *Hypercube Algebra: A Diagrammatic and Sentential Notation to Support Inferences in Logic* in *Proceedings of the 30th European Conference on Cognitive Ergonomics* (ACM, Edinburgh, United Kingdom, 2012), 100–104. ISBN: 978-1-4503-1786-3. doi:10.1145/2448136.2448157. http://doi.acm.org/10.1145/2448136.2448157.

57. McBryan, O. & Van de Velde, E. Hypercube Algorithms and Implementations. *SIAM Journal on Scientific and Statistical Computing* **8,** s227–s287 (1987).

58. Schlosser, M., Sintek, M., Decker, S. & Nejdl, W. HyperCuP Hypercubes, Ontologies, and Efficient Search on Peer-to-Peer Networks. **2530** (Oct. 2002).

59. Toce, A. *et al.* HyperD: Analysis and Performance Evaluation of a Distributed Hypercube Database (Sept. 2012).

60. Parhami, B. *Introduction to Parallel Processing: Algorithms and Processing* (Kluwer Academic Publishers, 2014).

61. Ferreira, A. *Issues in parallel computing with hypercube multiprocessors* 1994.

62. Bentley, J. L. Multidimensional Binary Search Trees Used for Associative Searching. *Commun. ACM* **18,** 509–517. ISSN: 0001-0782 (Sept. 1975).

63. Khot, S. *On the Unique Games Conjecture (Invited Survey)* in *2010 IEEE 25th Annual Conference on Computational Complexity* (June 2010), 99–121. doi:10.1109/CCC.2010.19.

64. Flocchini, P. *Optimal Elections in Labeled Hypercubes* 1996.

65. Anceaume, E., Ludinard, R., Ravoaja, A. & Brasileiro, F. *PeerCube: A Hypercube-Based P2P Overlay Robust against Collusion and Churn* in *Proceedings - 2nd IEEE International Conference on Self-Adaptive and Self-Organizing Systems, SASO 2008* (Oct. 2008), 15–24.

66. Harfoush, D. K. & Warrier, A. C. *Proximity Induced Labeling Schemes for Distributed Hash Tables* 2004.

67. Bang Choi, S. & Somani, A. A Parallel Algorithm for Embedding Complete Binary Trees in Faulty Hypercubes (Jan. 1998).

68. Rostami, H., Habibi, J. & Rahnama, A. *Semantic HyperCup* in. **9** (Feb. 2006), 223a –223a. ISBN: 0-7695-2507-5.

69. Bruls, M., Huizing, K. & van Wijk, J. *Squarified Treemaps* in *In Proceedings of the Joint Eurographics and IEEE TCVG Symposium on Visualization* (Press, 1999), 33–42.

70. Ninin, N. & Haucourt, E. The Boolean Algebra of Cubical Areas as a Tensor Product in the Category of Semilattices with Zero. **166** (Oct. 2014).

71. Go, J. T. The Terwilliger Algebra of the Hypercube. *Eur. J. Comb.* **23,** 399–429. ISSN: 0195-6698 (May 2002).

72. Saad, Y. & Schultz, M. H. Topological Properties of Hypercubes. *IEEE Trans. Comput.* **37,** 867–872. ISSN: 0018-9340 (July 1988).

73. Dogrusoz, U. Two–dimensional packing algorithms for layout of disconnected graphs. **143,** 147–158 (June 2002).

74. Romanuke, V. Uniform Sampling of the Infinite Noncooperative Game on Unit Hypercube and Reshaping Ultimately Multidimensional Matrices of Player's Payoff Values. **8** (July 2015).

75. Agarwal, N., Kindler, G., Kolla, A. & Trevisan, L. Unique Games on the Hypercube (May 2014).

76. Fletcher, G. H. L. & Sheth, H. A. *Unstructured Peer-to-Peer Networks: Topological Properties and Search Performance* in *Third International Joint Conference on Autonomous Agents and MUlti-Agent Systems. W6: Agents and Peerto-Peer Computing* (Springer, 2004), 14–27.

77. Slack, J. Visualization of embedded binary trees in the hypercube (May 2003).

78. Adjiman, P., Chatalic, P., Goasdoué, F., Rousset, M.-C. & Simon, L. Distributed Reasoning in a Peer-to-Peer Setting: Application to the Semantic Web. **25,** 269–314 (Jan. 2006).

79. Ngomo, A.-C. N. *et al. When to Reach for the Cloud: Using Parallel Hardware for Link Discovery* in *ESWC* (2013), 275–289.

80. Montani, C., Perego, R. & Scopigno, R. *Parallel Volume Visualization on a Hypercube Architecture* in *Proceedings of the 1992 Workshop on Volume Visualization* (ACM, Boston, Massachusetts, USA, 1992), 9–16. ISBN: 0-89791-527-5. doi:10.1145/147130.147139. http://doi.acm.org/10.1145/147130.147139.

81. Kosko, B. Addition as fuzzy mutual entropy. **73,** 273–284 (Oct. 1993).

82. Goodrich, M. & Tamassia, R. *Algorithm Design: Foundations, Analysis, and Internet Examples* ISBN: 9780471383659. `https://books.google.ca/books?id=gD8kAQAAIAAJ` (Wiley, 2002).

83. Mehlhorn, K., Näher, S. & Uhrig, C. *The LEDA platform for combinatorial and geometric computing* in *Automata, Languages and Programming* (eds Degano, P., Gorrieri, R. & Marchetti-Spaccamela, A.) (Springer Berlin Heidelberg, Berlin, Heidelberg, 1997), 7–16. ISBN: 978-3-540-69194-5.

84. Danzer, L. & Klee, V. Lengths of snakes in boxes. *Journal of Combinatorial Theory* **2,** 258 –265. ISSN: 0021-9800 (1967).

85. Tucovic, A. *Method for users to create and edit web page layouts* US Patent No. 9043698, Filed October 4th, 2012, Issued May 26th., 2015. 2015. `\url{https://patentimages.storage.googleapis.com/32/21/9a/ee5371070e7963/US9043698.pdf}`.

86. Tucovic, A. *Method for users to create and edit web page layouts* CA Patent No. CA2773152C, Filed September 28th, 2012, Issued June 7th., 2016. 2016.